

Final Code Report: NYT Surveillance Corpus Analysis

SIS/ITEC 724 Spring 2026. Professor Cogburn

Daniel Chavez

2026-04-21

Table of contents

Project Overview	2
Part 1: Environment Setup (CRISP-DM: Data Understanding)	3
Install R Packages	3
Load R Libraries	3
Verify and Set-up Python Environment	4
Part 2: Data Acquisition and Preparation (CRISP-DM: Data Understanding → Data Preparation)	4
Run the Fetch Pipeline	5
Pipeline Source Code	5
Load Final Corpus	45
Corpus Diff Report (Methodology Appendix)	49
Part 3: Preprocessing (CRISP-DM: Data Preparation)	49
Tokenize and Remove Stopwords	50
Part 4: Inductive Analysis (CRISP-DM: Modeling)	50
Word Frequencies	51
Bigram Analysis	52
TF-IDF Analysis	54
Part 5: Deductive Analysis (CRISP-DM: Modeling)	55
Bing Lexicon Sentiment Scoring	55
Longitudinal Sentiment Trend (RQ1.1)	56
Sentiment by Era (RQ3.2, RQ4.1)	57

VADER Sentiment in Python (RQ1.2)	58
Part 6: Elective Extensions (CRISP-DM: Modeling)	61
Quanteda Keyness (RQ3.1)	61
LDA Topic Modeling (RQ2.2)	64
GloVe Embeddings in R (RQ3.3, RQ4.2)	67
Word2Vec in Python (RQ3.3)	69
Part 7: Publication Volume, Event Windows, and Emotion (CRISP-DM: Evaluation)	70
Article Volume Over Time (RQ4.1, RQ4.2)	70
Event-Window Analysis (RQ4.1)	71
AI-Era Cutoff Sensitivity (RQ4.2)	73
NRC Emotion Analysis (RQ2.3)	75
Corpus-Size Normalization (Final-project methodology check)	75
Conclusion and Next Steps (CRISP-DM: Deployment)	76

Project Overview

This project conducts a longitudinal analysis of New York Times (NYT) surveillance coverage from 1995 through 2025, comparing two corpora: one of U.S. surveillance framing and one of People’s Republic of China (PRC) government surveillance framing across roughly 13,000 articles. The midterm built its corpora from ProQuest TDM Studio metadata exports, which surfaced only Title and Subject Terms because TDM Studio does not permit full-text export outside its AWS Jupyter environment. The final project rebuilds both corpora through the NYT Article Search and Archive APIs, which supply the richer `abstract` and `lead_paragraph` fields along with structured keyword arrays, print-edition metadata, and stable article URIs.

Analysis follows the CRISP-DM framework. This Final Code Report is a direct continuation of the midterm: Parts 3 through 7 retain the same analytical pipeline, now operating on the richer text surface, and the conclusion closes out each item from the midterm’s Remaining-for-Final checklist.

Disaggregated Research Questions (unchanged from midterm):

RQ1: Longitudinal sentiment trends

- RQ1.1: What is the overall trajectory of sentiment toward government surveillance in NYT coverage from 1995–2025? (Bing lexicon, loess trend)
- RQ1.2: Do VADER scores converge with dictionary-based results, validating the trend? (VADER triangulation)

RQ2: Dominant frames and themes

- RQ2.1: What are the most frequent terms and bigrams characterizing surveillance coverage? (word freq, n-grams, TF-IDF)
- RQ2.2: What latent topics emerge from LDA modeling, and how do topic proportions shift across eras? (LDA)
- RQ2.3: What emotional frames (NRC emotions) dominate coverage? (NRC lexicon)

RQ3: U.S. vs. PRC framing differences

- RQ3.1: Which words statistically distinguish U.S. from PRC coverage? (quanteda keyness/log-likelihood)
- RQ3.2: How do sentiment trajectories differ between the two corpora? (comparative sentiment)
- RQ3.3: Does the semantic neighborhood of “surveillance” differ across corpora? (GloVe + Word2Vec embeddings)

RQ4: Event-driven shifts

- RQ4.1: How did publication volume and sentiment shift around 9/11 (2001) and the Snowden revelations (2013)? (event-window analysis)
- RQ4.2: How did the emergence of AI-enabled surveillance correspond with changes in vocabulary and tone? (keyword tracking, embeddings, AI-era cutoff sensitivity testing)

Part 1: Environment Setup (CRISP-DM: Data Understanding)

The environment setup loads R packages across three text mining ecosystems (tm, tidytext, quanteda) alongside visualization, embeddings, and topic modeling libraries, and pulls in reticulate as the Python bridge that drives the later VADER and Word2Vec chunks.

Install R Packages

```
install.packages(c("tidytext", "tm", "quanteda", "quanteda.textplots",
                  "quanteda.textstats", "tidyverse", "scales",
                  "textdata", "topicmodels", "text2vec"))
```

Load R Libraries

```
library(tidyverse)
library(tidytext)
library(tm)
library(quanteda)
library(quanteda.textplots)
library(quanteda.textstats)
library(scales)
library(topicmodels)
```

Verify and Set-up Python Environment

```
reticulate::py_config()
```

```
import numpy as np
import warnings
warnings.filterwarnings('ignore')

import nltk
nltk.download("stopwords", quiet=True)
```

```
nltk.download("punkt", quiet=True)
```

```
nltk.download("punkt_tab", quiet=True)
```

```
nltk.download("vader_lexicon", quiet=True)
```

```
from nltk.corpus import stopwords
from nltk.sentiment.vader import SentimentIntensityAnalyzer
```

Part 2: Data Acquisition and Preparation (CRISP-DM: Data Understanding → Data Preparation)

The midterm assembled its corpus from ProQuest TDM Studio CSV exports; the final project rebuilds both corpora from scratch through the NYT APIs to recover the full metadata schema. Two paths run in parallel: the Archive API serves as the primary source, returning every NYT

article for a given month in one call, while the Article Search API runs the Boolean queries server-side via NYT's Lucene engine as a validation layer. The pipeline reconciles both sources on the `uri` key, joins the unified corpus against the midterm's ProQuest CSVs to preserve Subject Terms, and loads the result into R as `sample_corpus` with the same variable names the downstream analysis expects. The Python pipeline modules (`nyt_common.py`, `nyt_archive_fetch.py`, `nyt_article_search_fetch.py`, `nyt_unify_and_merge.py`) live alongside this report in the project directory.

Run the Fetch Pipeline

The three fetch scripts are idempotent and resumable. Each call against the NYT API is paced at 12.1 seconds to respect the 5 requests per minute limit, and the daily budget caps at 495 requests under the 500-per-day ceiling. The probe chunk below runs one month (2013-06, peak Snowden era) as a Boolean-recall sanity check before the full crawl.

Pipeline Source Code

The four Python modules below constitute the fetch pipeline. They are included here as non-executing chunks so every line of the data-preparation code is visible in this report. The modules are saved separately as `.py` files in the project directory and run from the command line as shown in the bash chunks further down. The CSVs loaded by the next section are the output of these modules.

```
"""
nyt_common.py: shared utilities for both NYT fetchers.

ITEC 724 final project. CRISP-DM phase: Data Understanding to Data Preparation.

Provides field extractors, a Lucene-subset Boolean engine, era tagging,
checkpoint I/O, and the merge logic that reconciles Archive with Article
Search and joins both against the ProQuest corpus.

Schema notes, verified against the current developer-portal OpenAPI specs:
both Archive and Article Search return the same Article shape, with two
differences worth flagging. First, Article Search renamed `news_desk` to
`desk` in the April 2025 revision; Archive still uses `news_desk`. Second,
Article Search dropped `_id` from its schema while Archive retains it, so
`uri` (format "nyt://article/...") is the only stable identifier shared by
both APIs and must be used as the dedup key. Earlier drafts of this module
speculated about parallel renames (headline.default, Article.wordCount,
bylines[]); the specs do not confirm those, and the legacy names win in
```

practice. The extractors below keep defensive fallbacks anyway.

Source filtering differs by API: Article Search filters server-side via ``fq=source.vernacular:"The New York Times"`` because the response schema omits the ``source`` field entirely. Archive includes ``source`` in every doc, so the filter runs client-side against `VALID_SOURCES`.

Boolean engine: the parser supports quoted phrases, unquoted terms with word-boundary matching, AND/OR, and parenthesized subexpressions. It does not support NOT, proximity operators, wildcards, or field-qualified queries; none appear in the surveillance queries. Unquoted ``NSA`` matches "NSA surveillance" but not "unsanitary" or "CNSA"; case-insensitive throughout.

Matching surface: headline, abstract, lead_paragraph, and concatenated keyword values. This mirrors the Article Search API's default ``q=` scan` (body, headline, byline) while remaining feasible against Archive metadata, which carries no full-body text.

"""

```
from __future__ import annotations
```

```
import json
```

```
import logging
```

```
import re
```

```
from dataclasses import dataclass
```

```
from datetime import date, datetime
```

```
from pathlib import Path
```

```
from typing import Any, Union
```

```
import pandas as pd
```

```
from rapidfuzz import fuzz
```

```
# -----
```

```
# Era tagging - mirrors the midterm segmentation
```

```
# -----
```

```
def tag_era(pub_date: date) -> str:
```

```
    """Assign each article to one of four eras used in midterm analysis."""
```

```
    if pub_date < date(2001, 9, 11):
```

```
        return "pre_9_11"
```

```
    if pub_date < date(2013, 6, 5): # Snowden disclosures begin
```

```

        return "post_9_11_pre_snowden"
    if pub_date < date(2020, 1, 1):
        return "snowden_era"
    return "post_2020"

# -----
# Defensive field extractors - handles both pre- and post-April-2025 shapes
# -----

def _first(*vals: Any) -> Any:
    """Return the first non-empty value, or '' if all are empty/None."""
    for v in vals:
        if v is None:
            continue
        if isinstance(v, str) and not v.strip():
            continue
        if isinstance(v, (list, dict)) and not v:
            continue
        return v
    return ""

def extract_headline(doc: dict) -> str:
    """headline.default (new) or headline.main (legacy)."""
    h = doc.get("headline") or {}
    if isinstance(h, str):
        return h
    return _first(h.get("default"), h.get("main"), h.get("seo"), "")

def extract_byline(doc: dict) -> str:
    """bylines[].renderedRepresentation (new) or byline.original (legacy)."""
    bylines = doc.get("bylines")
    if isinstance(bylines, list) and bylines:
        rendered = (bylines[0] or {}).get("renderedRepresentation")
        if rendered:
            return rendered
    if isinstance(bylines, dict):
        rendered = bylines.get("renderedRepresentation")
        if rendered:
            return rendered

```

```

b = doc.get("byline") or {}
if isinstance(b, dict):
    return _first(b.get("original"), "")
return ""

def extract_word_count(doc: dict) -> int:
    """Article.wordCount (new) or word_count (legacy)."""
    direct = doc.get("word_count")
    if direct is not None:
        try:
            return int(direct)
        except (ValueError, TypeError):
            pass
    article = doc.get("Article") or doc.get("article") or {}
    if isinstance(article, dict):
        wc = article.get("wordCount")
        if wc is not None:
            try:
                return int(wc)
            except (ValueError, TypeError):
                pass
    return 0

def extract_section(doc: dict) -> str:
    """section.displayName / section.name (new) or section_name (legacy)."""
    direct = doc.get("section_name")
    if direct:
        return direct
    section = doc.get("section")
    if isinstance(section, dict):
        return _first(section.get("displayName"), section.get("name"), "")
    if isinstance(section, str):
        return section
    return ""

def extract_desk(doc: dict) -> str:
    """
    `desk` (Article Search, April 2025 rename) or `news_desk` (Archive, still legacy).
    Both APIs mean the same thing: Foreign, Metro, Sports, etc.

```

```

"""
return _first(doc.get("news_desk"), doc.get("desk"), "")

def extract_keywords(doc: dict) -> list:
    """
    `keywords[]` with {name, value, rank} - confirmed present in both current
    Archive and Article Search specs. `timesTags` fallback retained defensively
    in case a future revision actually migrates response field names.
    """
    kws = doc.get("keywords")
    if isinstance(kws, list) and kws:
        return kws
    tags = doc.get("timesTags")
    if isinstance(tags, list) and tags:
        return tags
    return []

def extract_keyword_values(doc: dict) -> str:
    """Concatenate keyword values into a single string for Boolean matching."""
    kws = extract_keywords(doc)
    parts = []
    for kw in kws:
        if not isinstance(kw, dict):
            continue
        # Legacy: {"name": "...", "value": "...", "rank": ...}
        # New (speculative): {"displayName": "...", "type": "...", "vernacular": "..."}
        v = _first(kw.get("value"), kw.get("displayName"), kw.get("vernacular"), "")
        if v:
            parts.append(str(v))
    return " ".join(parts)

def extract_web_url(doc: dict) -> str:
    return _first(doc.get("web_url"), doc.get("url"), "")

def extract_id(doc: dict) -> str:
    """
    Unique identifier. Prefer `uri` (format: nyt://article/...) because it's
    the ONLY ID field documented in both the current Archive spec and the

```

```

current Article Search spec. `_id` still appears on Archive responses but
was dropped from the Article Search schema in the April 2025 revision,
so using `_id` as the dedup key would misclassify every Article Search
row as a new article and corrupt the unifier.
"""

return _first(doc.get("uri"), doc.get("_id"), "")

def build_match_surface(doc: dict) -> str:
    """
    Concatenate the text fields the Boolean engine scans.
    Per final-project design: headline + abstract + lead_paragraph + keyword values.
    This mirrors the Article Search API's default `q=` scan surface while
    remaining feasible against Archive metadata (no full-body text).
    """
    parts = [
        extract_headline(doc),
        doc.get("abstract") or doc.get("summary") or "",
        doc.get("lead_paragraph") or "",
        extract_keyword_values(doc),
    ]
    return " ".join(p for p in parts if p)

def flatten_doc(doc: dict, corpus: str, source_api: str) -> dict:
    """
    Flatten one API document into the unified row schema.

    source_api: "archive" or "article_search" - records provenance so the
    unified corpus can be split back out for validation.
    """
    pub_date_raw = doc.get("pub_date") or doc.get("firstPublished") or ""
    try:
        # NYT Archive returns dates like "1995-01-01T05:00:00+0000" - the +0000
        # tz offset without a colon breaks fromisoformat() on Python < 3.11.
        # Normalize by inserting the colon.
        import re
        normalized = re.sub(r'([+-]\d{2})(\d{2})$', r'\1:\2', pub_date_raw.replace("Z", "+00"))
        pub_dt = datetime.fromisoformat(normalized)
        pub_date_iso = pub_dt.date().isoformat()
        era = tag_era(pub_dt.date())
    except (ValueError, TypeError, AttributeError):

```

```

    pub_date_iso = ""
    era = ""

    return {
        "source_api": source_api,
        "corpus": corpus,
        "era": era,
        "pub_date": pub_date_iso,
        "headline": extract_headline(doc),
        "abstract": doc.get("abstract") or doc.get("summary") or "",
        "lead_paragraph": doc.get("lead_paragraph") or "",
        "snippet": doc.get("snippet") or "",
        "keywords_json": json.dumps(extract_keywords(doc), ensure_ascii=False),
        "section_name": extract_section(doc),
        "news_desk": extract_desk(doc),
        "byline": extract_byline(doc),
        "word_count": extract_word_count(doc),
        "web_url": extract_web_url(doc),
        # `uri` is the authoritative stable ID across both APIs; we keep the
        # column named `_id` for backward-compat with the midterm schema but
        # it actually holds `uri` when available (Archive may also set `_id`).
        "_id": extract_id(doc),
        # Print-edition fields - present in both specs, useful for the paper:
        # article prominence (front page vs. A12) is a known signal for
        # surveillance-coverage intensity over time.
        "print_page": doc.get("print_page") or "",
        "print_section": doc.get("print_section") or "",
        "type_of_material": doc.get("type_of_material") or "",
        "document_type": doc.get("document_type") or "",
    }
}

UNIFIED_OUTPUT_COLUMNS = [
    "source_api", "match_source", "corpus", "era", "pub_date", "headline",
    "abstract", "lead_paragraph", "snippet", "keywords_json", "section_name",
    "news_desk", "byline", "word_count", "web_url", "_id",
    "print_page", "print_section", "type_of_material", "document_type",
    "proquest_date", "proquest_title", "proquest_subject_terms",
]

# -----

```


Note: no implicit AND. Every conjunction must be explicit. The surveillance queries always supply AND/OR explicitly, so this is a feature not a bug.

```
"""
```

```
def __init__(self, tokens: list[str]):
    self.tokens = tokens
    self.pos = 0

def _peek(self) -> str | None:
    return self.tokens[self.pos] if self.pos < len(self.tokens) else None

def _consume(self) -> str:
    tok = self.tokens[self.pos]
    self.pos += 1
    return tok

def parse(self):
    node = self._or_expr()
    if self.pos != len(self.tokens):
        raise ValueError(f"Unparsed trailing tokens at pos {self.pos}: {self.tokens[self.pos:]}")
    return node

def _or_expr(self):
    left = self._and_expr()
    parts = [left]
    while self._peek() == "OR":
        self._consume()
        parts.append(self._and_expr())
    if len(parts) == 1:
        return left
    return ("or", parts)

def _and_expr(self):
    left = self._atom()
    parts = [left]
    while self._peek() == "AND":
        self._consume()
        parts.append(self._atom())
    if len(parts) == 1:
        return left
    return ("and", parts)
```

```

def _atom(self):
    tok = self._peek()
    if tok is None:
        raise ValueError("Unexpected end of query")
    if tok == "(":
        self._consume()
        node = self._or_expr()
        closer = self._peek()
        if closer != ")":
            raise ValueError(f"Expected ')' but got {closer!r}")
        self._consume()
        return node
    if tok in ("AND", "OR", ")"):
        raise ValueError(f"Unexpected operator {tok!r} at pos {self.pos}")
    self._consume()
    if tok.startswith("'") and tok.endswith("'"):
        return ("phrase", tok[1:-1])
    return ("term", tok)

def parse_query(query: str):
    """Parse a Boolean query string into an AST."""
    tokens = _tokenize(query)
    if not tokens:
        raise ValueError("Empty query")
    return _Parser(tokens).parse()

def _compile_node(node, cache: dict) -> "re.Pattern":
    """
    Compile AST leaves to regex patterns with word-boundary semantics.
    Caches by node value so repeated terms across a large corpus are fast.
    AND/OR nodes don't compile to regex; they're handled in match_node.
    """
    kind = node[0]
    if kind == "term":
        key = ("term", node[1].lower())
        if key not in cache:
            cache[key] = re.compile(r'\b' + re.escape(node[1]) + r'\b', re.IGNORECASE)
        return cache[key]
    if kind == "phrase":
        key = ("phrase", node[1].lower())

```

```

    if key not in cache:
        # Collapse internal whitespace in the query phrase to \s+ so that
        # "mass surveillance" matches "mass  surveillance" and "mass\nsurveillance".
        parts = [re.escape(w) for w in node[1].split()]
        if not parts:
            # Degenerate: empty quoted string. Match nothing.
            cache[key] = re.compile(r'(!.*)')
        else:
            pattern = r'\b' + r'\s+'.join(parts) + r'\b'
            cache[key] = re.compile(pattern, re.IGNORECASE)
    return cache[key]
raise ValueError(f"Not a leaf node: {node}")

def match_node(node, text: str, cache: dict) -> bool:
    """Evaluate an AST node against a text string."""
    kind = node[0]
    if kind in ("term", "phrase"):
        return bool(_compile_node(node, cache).search(text))
    if kind == "and":
        return all(match_node(child, text, cache) for child in node[1])
    if kind == "or":
        return any(match_node(child, text, cache) for child in node[1])
    raise ValueError(f"Unknown node kind: {kind}")

# -----
# Checkpointing
# -----

def load_checkpoint(path: Path) -> set[str]:
    if not path.exists():
        return set()
    completed = set()
    with path.open("r", encoding="utf-8") as fh:
        for line in fh:
            line = line.strip()
            if not line:
                continue
            try:
                completed.add(json.loads(line)["window_key"])
            except (json.JSONDecodeError, KeyError):

```

```

        continue
    return completed

def append_checkpoint(path: Path, window_key: str, result_count: int, note: str = "") -> None:
    rec = {
        "window_key": window_key,
        "result_count": result_count,
        "note": note,
        "timestamp": datetime.utcnow().isoformat() + "Z",
    }
    with path.open("a", encoding="utf-8") as fh:
        fh.write(json.dumps(rec) + "\n")

def append_rows(csv_path: Path, rows: list[dict]) -> None:
    if not rows:
        return
    df = pd.DataFrame(rows)
    header = not csv_path.exists()
    df.to_csv(csv_path, mode="a", header=header, index=False, encoding="utf-8")

def append_log(log_path: Path, rec: dict) -> None:
    df = pd.DataFrame([rec])
    header = not log_path.exists()
    df.to_csv(log_path, mode="a", header=header, index=False, encoding="utf-8")

# -----
# Log response shape once (diagnostic)
# -----

def log_response_shape_once(sample_doc: dict, state_ref, log: logging.Logger) -> None:
    """Emit a one-shot diagnostic describing which response field names arrived."""
    if getattr(state_ref, "response_shape_logged", False):
        return
    keys = sorted(sample_doc.keys())
    has_new_headline = isinstance(sample_doc.get("headline"), dict) and \
        "default" in sample_doc["headline"]
    has_new_byline = "bylines" in sample_doc
    has_new_section = isinstance(sample_doc.get("section"), dict)

```

```

has_new_wordcount = isinstance(sample_doc.get("Article"), dict) and \
    "wordCount" in sample_doc["Article"]
log.info("=" * 70)
log.info("FIRST-FETCH RESPONSE SHAPE DIAGNOSTIC")
log.info(f"  top-level keys: {keys}")
log.info(f"  uses new schema? headline.default={has_new_headline}, "
        f"bylines[]={has_new_byline}, section.obj={has_new_section}, "
        f"Article.wordCount={has_new_wordcount}")
log.info("=" * 70)
state_ref.response_shape_logged = True

# -----
# Merge with ProQuest CSVs (two-pass: exact date+title, then fuzzy)
# -----

def normalize_title(s: str) -> str:
    if not isinstance(s, str):
        return ""
    return s.lower().strip()

def merge_with_proquest(
    api_df: pd.DataFrame,
    proquest_csv: Path,
    corpus: str,
    out_csv: Path,
    unmatched_csv: Path,
    log: logging.Logger,
    fuzzy_threshold: int = 85,
    borderline_csv: Path | None = None,
    borderline_score_ceiling: int = 95,
) -> None:
    """
    Two-pass match of an API-sourced DataFrame against a ProQuest CSV.
    Writes the merged output and unmatched ProQuest rows to disk.

    api_df: pre-loaded DataFrame (already has source_api column from flatten_doc)

    borderline_csv: if provided, fuzzy matches scoring in
    [fuzzy_threshold, borderline_score_ceiling) are logged here with both
    titles side-by-side for spot-checking. A 90%-similar headline might be

```

the same article (print vs. web headline) or two different articles on the same date. Reviewing the borderline file lets you catch false-positive promotions without hand-inspecting every match. Recommended for methodology appendix.

```
"""
```

```
log.info(f"=== Merging corpus {corpus} ({len(api_df)} API rows) with {proquest_csv.name}")
```

```
pq_df = pd.read_csv(proquest_csv, dtype=str, keep_default_na=False)
```

```
pq_df = pq_df.rename(columns={
```

```
    "Date": "proquest_date",
```

```
    "Title": "proquest_title",
```

```
    "Subject Terms": "proquest_subject_terms",
```

```
})
```

```
pq_df["proquest_date_iso"] = pd.to_datetime(
```

```
    pq_df["proquest_date"], errors="coerce"
```

```
).dt.date.astype(str)
```

```
api_df = api_df.copy()
```

```
api_df["_title_norm"] = api_df["headline"].map(normalize_title)
```

```
pq_df["_title_norm"] = pq_df["proquest_title"].map(normalize_title)
```

```
exact = api_df.merge(
```

```
    pq_df,
```

```
    left_on=["pub_date", "_title_norm"],
```

```
    right_on=["proquest_date_iso", "_title_norm"],
```

```
    how="outer",
```

```
    indicator=True,
```

```
)
```

```
matched_mask = exact["_merge"] == "both"
```

```
api_only_mask = exact["_merge"] == "left_only"
```

```
pq_only_after_exact = exact[exact["_merge"] == "right_only"].copy()
```

```
log.info(
```

```
    f" Pass 1 (exact): matched={matched_mask.sum()}, "
```

```
    f"api_only={api_only_mask.sum()}, pq_unmatched={len(pq_only_after_exact)}"
```

```
)
```

```
api_unmatched = exact[exact["_merge"] == "left_only"].copy()
```

```
api_unmatched_by_date: dict[str, list[int]] = {}
```

```
for idx, row in api_unmatched.iterrows():
```

```
    api_unmatched_by_date.setdefault(row["pub_date"], []).append(idx)
```

```

fuzzy_match_rows = []
still_unmatched_pq = []
borderline_log = [] # rows with score in [threshold, ceiling)

for pq_idx, pq_row in pq_only_after_exact.iterrows():
    candidates = api_unmatched_by_date.get(pq_row["proquest_date_iso"], [])
    best_score = 0
    best_api_idx = None
    for api_idx in candidates:
        score = fuzz.token_set_ratio(
            pq_row["_title_norm"],
            api_unmatched.loc[api_idx, "_title_norm"],
        )
        if score > best_score:
            best_score = score
            best_api_idx = api_idx
    if best_score >= fuzzy_threshold and best_api_idx is not None:
        fuzzy_match_rows.append((pq_idx, best_api_idx))
        api_unmatched_by_date[pq_row["proquest_date_iso"]].remove(best_api_idx)
        if best_score < borderline_score_ceiling:
            borderline_log.append({
                "pub_date": pq_row["proquest_date_iso"],
                "score": best_score,
                "proquest_title": pq_row["proquest_title"],
                "api_title": api_unmatched.loc[best_api_idx, "headline"],
                "api_uri": api_unmatched.loc[best_api_idx, "_id"],
                "api_source": api_unmatched.loc[best_api_idx, "source_api"],
                "proquest_subject_terms": pq_row.get("proquest_subject_terms", ""),
            })
        else:
            still_unmatched_pq.append(pq_idx)

log.info(
    f" Pass 2 (fuzzy >={fuzzy_threshold}): promoted={len(fuzzy_match_rows)} "
    f"(of which {len(borderline_log)} scored <{borderline_score_ceiling}), "
    f"remaining pq_only={len(still_unmatched_pq)}"
)

if borderline_log and borderline_csv is not None:
    pd.DataFrame(borderline_log).sort_values(["pub_date", "score"]).to_csv(
        borderline_csv, index=False, encoding="utf-8"
    )

```

```

log.info(f" Wrote borderline fuzzy matches: {borderline_csv} ({len(borderline_log)})

matched_df = exact[matched_mask].copy()
matched_df["match_source"] = "both"

promoted_api_idx = {pair[1] for pair in fuzzy_match_rows}
api_only_df = api_unmatched.loc[~api_unmatched.index.isin(promoted_api_idx)].copy()
api_only_df["match_source"] = "api_only"
for col in ("proquest_date", "proquest_title", "proquest_subject_terms"):
    if col not in api_only_df.columns:
        api_only_df[col] = ""
    api_only_df[col] = api_only_df[col].fillna("")

fuzzy_df_rows = []
for pq_idx, api_idx in fuzzy_match_rows:
    api_row = api_unmatched.loc[api_idx].to_dict()
    pq_row = pq_only_after_exact.loc[pq_idx]
    api_row["proquest_date"] = pq_row["proquest_date"]
    api_row["proquest_title"] = pq_row["proquest_title"]
    api_row["proquest_subject_terms"] = pq_row["proquest_subject_terms"]
    api_row["match_source"] = "both"
    fuzzy_df_rows.append(api_row)
fuzzy_df = pd.DataFrame(fuzzy_df_rows) if fuzzy_df_rows else pd.DataFrame()

pq_only_df = pq_only_after_exact.loc[still_unmatched_pq].copy()
pq_only_df["match_source"] = "proquest_only"
for col in UNIFIED_OUTPUT_COLUMNS:
    if col not in pq_only_df.columns:
        pq_only_df[col] = ""
pq_only_df["corpus"] = corpus
pq_only_df["source_api"] = ""
pq_only_df["era"] = pq_only_df["proquest_date_iso"].apply(
    lambda s: tag_era(date.fromisoformat(s)) if s and s != "NaT" else ""
)

merged = pd.concat(
    [matched_df, api_only_df, fuzzy_df, pq_only_df],
    ignore_index=True,
    sort=False,
)

for col in UNIFIED_OUTPUT_COLUMNS:

```

```

        if col not in merged.columns:
            merged[col] = ""
    merged = merged[UNIFIED_OUTPUT_COLUMNS]
    merged.to_csv(out_csv, index=False, encoding="utf-8")
    log.info(f" Wrote merged: {out_csv} ({len(merged)} rows)")

    if still_unmatched_pq:
        inspect = pq_only_after_exact.loc[still_unmatched_pq, [
            "proquest_date", "proquest_title", "proquest_subject_terms"
        ]]
        inspect.to_csv(unmatched_csv, index=False, encoding="utf-8")
        log.info(f" Wrote unmatched ProQuest rows: {unmatched_csv} ({len(inspect)})")

# -----
# Unified-corpus merger (Archive + Article Search)
# -----

def unify_corpora(
    archive_csv: Path,
    article_search_csv: Path,
    corpus: str,
    out_csv: Path,
    diff_csv: Path,
    log: logging.Logger,
) -> pd.DataFrame:
    """
    Merge Archive-sourced and Article-Search-sourced CSVs on _id.

    Writes:
        out_csv - unified corpus with source_api set to 'archive', 'article_search', or 'both'
        diff_csv - articles present in only one source, for methodology appendix

    Returns the unified DataFrame so the caller can pipe it into merge_with_proquest.
    """
    log.info(f"=== Unifying {corpus}: {archive_csv.name} + {article_search_csv.name} ===")

    have_archive = archive_csv.exists()
    have_as = article_search_csv.exists()

    if have_archive and not have_as:
        log.info(" Only Archive CSV present; passing through.")

```

```

    df = pd.read_csv(archive_csv, dtype=str, keep_default_na=False)
    df.to_csv(out_csv, index=False, encoding="utf-8")
    return df
if have_as and not have_archive:
    log.info(" Only Article Search CSV present; passing through.")
    df = pd.read_csv(article_search_csv, dtype=str, keep_default_na=False)
    df.to_csv(out_csv, index=False, encoding="utf-8")
    return df
if not have_archive and not have_as:
    raise FileNotFoundError(f"Neither {archive_csv} nor {article_search_csv} exists.")

arch = pd.read_csv(archive_csv, dtype=str, keep_default_na=False)
as_ = pd.read_csv(article_search_csv, dtype=str, keep_default_na=False)

arch_ids = set(arch["_id"]) - {""}
as_ids = set(as_["_id"]) - {""}
in_both = arch_ids & as_ids
arch_only = arch_ids - as_ids
as_only = as_ids - arch_ids

log.info(f" archive_only={len(arch_only)}, article_search_only={len(as_only)}, both={len(in_both)}")

# For rows in both: keep the Archive row (it has full coverage including
# older years Article Search might miss) and mark source_api='both'.
arch_marked = arch.copy()
arch_marked.loc[arch_marked["_id"].isin(in_both), "source_api"] = "both"
# Archive-only rows keep source_api='archive' from flatten_doc.

# Add Article-Search-only rows.
as_only_df = as_[as_["_id"].isin(as_only)].copy()
# source_api is already 'article_search' from flatten_doc.

unified = pd.concat([arch_marked, as_only_df], ignore_index=True, sort=False)
unified.to_csv(out_csv, index=False, encoding="utf-8")
log.info(f" Wrote unified corpus: {out_csv} ({len(unified)} rows)")

# Diff report for methodology appendix
diff_rows = []
for _id in sorted(arch_only):
    row = arch[arch["_id"] == _id].iloc[0]
    diff_rows.append({
        "_id": _id, "only_in": "archive",

```

```

        "pub_date": row.get("pub_date", ""),
        "headline": row.get("headline", ""),
    })
for _id in sorted(as_only):
    row = as_[as["_id"] == _id].iloc[0]
    diff_rows.append({
        "_id": _id, "only_in": "article_search",
        "pub_date": row.get("pub_date", ""),
        "headline": row.get("headline", ""),
    })
if diff_rows:
    pd.DataFrame(diff_rows).to_csv(diff_csv, index=False, encoding="utf-8")
    log.info(f" Wrote diff report: {diff_csv} ({len(diff_rows)} rows)")

return unified

```

```

"""
nyt_archive_fetch.py: primary corpus builder using the NYT Archive API.

ITEC 724 final project. CRISP-DM phase: Data Understanding to Data Preparation.

The Archive API returns every NYT article for a given month in one call,
going back to 1851. The fetcher pulls one month at a time (372 calls for
1995 through 2025), gzips the raw JSON to disk for reproducibility, then
runs the Boolean engine locally against headline, abstract, lead_paragraph,
and keyword values to assemble the U.S. and PRC surveillance corpora.

Archive is primary because it sidesteps the Article Search pagination cap
of 1,000 results per query window and the 10,000-hit reporting ceiling.
Raw dumps on disk mean Boolean query revisions re-filter for free. Archive
is not sufficient on its own: the local engine lacks the authority of NYT's
Lucene semantics, so Article Search runs separately as a validation layer.

Storage: 372 gzipped JSON files, roughly 1 to 5 MB each, totaling about 1 GB.
Rate limits: the current developer portal does not publish Archive-specific
limits; the script assumes the shared 5/min floor and sleeps 12.1 seconds
between calls, so a full crawl takes about 75 minutes of wall time.
"""

from __future__ import annotations

import argparse

```

```

import gzip
import json
import logging
import os
import sys
import time
from dataclasses import dataclass, field
from datetime import date, datetime
from pathlib import Path

import pandas as pd
import requests
from dotenv import load_dotenv

# Import from sibling module
sys.path.insert(0, str(Path(__file__).parent))
from nyt_common import (
    append_checkpoint, append_log, append_rows, build_match_surface,
    flatten_doc, load_checkpoint, log_response_shape_once, match_node,
    parse_query,
)

# -----
# Configuration
# -----

ARCHIVE_URL = "https://api.nytimes.com/svc/archive/v1/{year}/{month}.json"

SLEEP_BETWEEN_REQUESTS = 12.1
DAILY_REQUEST_BUDGET = 495          # shared 5/min floor is the real constraint
MAX_RETRIES_PER_CALL = 5
BACKOFF_BASE = 2.0

# Only keep articles where source == "The New York Times", matching the
# ProQuest filter. AP/Reuters wire reprints are excluded.
VALID_SOURCES = {"The New York Times"}

# -----
# Boolean queries - identical to Article Search, since the semantics must match
# -----

```

```

US_QUERY = (
    '("government surveillance" OR "mass surveillance" OR "domestic surveillance" '
    'OR "electronic surveillance" OR "surveillance program" OR "surveillance state" '
    'OR "warrantless surveillance" OR "warrantless wiretapping" '
    'OR "National Security Agency" OR "NSA surveillance" OR "NSA spying" OR "NSA program" '
    'OR "Patriot Act" OR "USA FREEDOM Act" OR "Foreign Intelligence Surveillance" '
    'OR "Section 215" OR "Section 702" OR "bulk collection" OR "metadata collection" '
    'OR "Edward Snowden" OR "Snowden revelations" OR "Total Information Awareness" '
    'OR "Clipper Chip" OR "Carnivore" OR "ECHELON" OR "dragnet surveillance" '
    'OR "domestic spying" OR "terrorist surveillance program" OR "Clearview AI" '
    'OR "Stingray surveillance" OR "biometric surveillance" '
    'OR ("facial recognition" AND police) OR "predictive policing")'
)

PRC_QUERY = (
    '((China OR Chinese OR Beijing) AND ('
    'surveillance OR "mass surveillance" OR "government surveillance" '
    'OR "social credit system" OR "social credit score" '
    'OR "Skynet" OR "Sharp Eyes" OR "Golden Shield" OR "digital authoritarianism" '
    'OR ("Xinjiang" AND (surveillance OR monitoring OR internment)) '
    'OR ("Uyghur" AND (surveillance OR monitoring OR tracking)) '
    'OR ("public security" AND (surveillance OR monitoring OR cameras)) '
    'OR ("CCTV" AND surveillance) OR "video surveillance" OR "camera network" '
    'OR ("grid management" AND surveillance) OR "DNA database" '
    'OR "biometric surveillance" OR "gait recognition" OR "emotion recognition" '
    'OR ("internet censorship" AND surveillance) '
    'OR ("cyber sovereignty" AND surveillance) '
    'OR "Integrated Joint Operations Platform" OR "IJOP" '
    'OR "surveillance state" OR "police state" '
    'OR "surveillance technology" OR "surveillance system" '
    'OR (("spying" OR "espionage") AND domestic)'
    '))'
)

# -----
# Archive fetch state
# -----

@dataclass
class ArchiveState:
    requests_today: int = 0

```

```

    completed_windows: set = field(default_factory=set)
    response_shape_logged: bool = False

    def can_request(self, n: int = 1) -> bool:
        return self.requests_today + n <= DAILY_REQUEST_BUDGET

class BudgetExhausted(Exception):
    pass

# -----
# Archive API call
# -----

def fetch_archive_month(
    year: int,
    month: int,
    api_key: str,
    raw_dir: Path,
    state: ArchiveState,
    log: logging.Logger,
) -> list[dict]:
    """
    Fetch one month from the Archive API.

    Writes the raw response (gzipped) to raw_dir/YYYY-MM.json.gz and returns
    the list of docs. If the gzipped file already exists, reads from disk
    instead of re-calling the API (idempotent replay).
    """
    raw_path = raw_dir / f"{year:04d}-{month:02d}.json.gz"
    if raw_path.exists():
        log.info(f"[{year}-{month:02d}] reading from cache: {raw_path.name}")
        with gzip.open(raw_path, "rt", encoding="utf-8") as fh:
            data = json.load(fh)
        return (data.get("response") or {}).get("docs") or []

    url = ARCHIVE_URL.format(year=year, month=month)
    params = {"api-key": api_key}

    last_error: Exception | None = None
    for attempt in range(MAX_RETRIES_PER_CALL):

```

```

if not state.can_request():
    raise BudgetExhausted()

state.requests_today += 1
try:
    resp = requests.get(url, params=params, timeout=120) # larger timeout; payload
except requests.RequestException as e:
    last_error = e
    backoff = BACKOFF_BASE * (2 ** attempt)
    log.warning(f"[{year}--{month:02d}] network error (attempt {attempt + 1}): {e}. Sleeping {backoff}s.")
    time.sleep(backoff)
    continue

if resp.status_code == 429:
    backoff = BACKOFF_BASE * (2 ** attempt)
    log.warning(f"[{year}--{month:02d}] HTTP 429. Sleeping {backoff}s.")
    time.sleep(backoff)
    continue

if resp.status_code >= 500:
    backoff = BACKOFF_BASE * (2 ** attempt)
    log.warning(f"[{year}--{month:02d}] HTTP {resp.status_code}. Sleeping {backoff}s.")
    time.sleep(backoff)
    continue

if resp.status_code != 200:
    resp.raise_for_status()

data = resp.json()
docs = (data.get("response") or {}).get("docs") or []

# Write gzipped raw dump for reproducibility.
raw_dir.mkdir(parents=True, exist_ok=True)
with gzip.open(raw_path, "wt", encoding="utf-8") as fh:
    json.dump(data, fh, ensure_ascii=False)
size_mb = raw_path.stat().st_size / 1_048_576
log.info(f"[{year}--{month:02d}] fetched {len(docs)} docs, wrote {raw_path.name} ({size_mb} MB)")

if docs and not state.response_shape_logged:
    log_response_shape_once(docs[0], state, log)

time.sleep(SLEEP_BETWEEN_REQUESTS)

```

```

        return docs

    raise RuntimeError(f"Exceeded retries for {year}-{month:02d}: {last_error}")

# -----
# Filter one month's docs into both corpora
# -----

def filter_and_tag(
    docs: list[dict],
    us_ast,
    prc_ast,
    query_cache: dict,
    log: logging.Logger,
) -> tuple[list[dict], list[dict], dict]:
    """
    Run the Boolean engine against each doc and return (us_rows, prc_rows, stats).
    Articles matching both queries appear in both outputs; corpus-level dedup
    happens downstream, not here.
    """
    us_rows = []
    prc_rows = []
    stats = {"total": len(docs), "kept_source": 0, "us_match": 0, "prc_match": 0, "either": 0}

    for doc in docs:
        # Source filter: only The New York Times proper.
        src = doc.get("source") or ""
        if src not in VALID_SOURCES:
            continue
        stats["kept_source"] += 1

        surface = build_match_surface(doc)
        if not surface:
            continue

        us_hit = match_node(us_ast, surface, query_cache)
        prc_hit = match_node(prc_ast, surface, query_cache)

        if us_hit:
            us_rows.append(flatten_doc(doc, "US", "archive"))
            stats["us_match"] += 1

```

```

        if prc_hit:
            prc_rows.append(flatten_doc(doc, "PRC", "archive"))
            stats["prc_match"] += 1
        if us_hit or prc_hit:
            stats["either"] += 1

    return us_rows, prc_rows, stats

# -----
# Main orchestration
# -----

def iter_year_months(start_year: int, end_year: int):
    for year in range(start_year, end_year + 1):
        for month in range(1, 13):
            yield year, month

# -----
# 2020+ coverage probe
# -----
# The current Archive spec lists the supported year range as 1851-2019. Two
# readings are possible: the doc is stale, or the cap is enforced server-side.
# The probe runs one diagnostic call before the main crawl so a 2020+ gap
# surfaces immediately. If it fails, the script pivots to an Article-Search
# fallback for 2020 through 2025 rather than discovering the problem 25 years
# of data later.

def probe_2020_coverage(api_key: str, state, log) -> bool:
    """
    Test whether the Archive API serves post-2019 data by hitting one month
    from 2024. Returns True if the response is populated, False otherwise.
    The probe consumes one request from the daily budget and doubles as a
    connectivity and authentication check.
    """
    log.info("=" * 70)
    log.info("ARCHIVE 2020+ COVERAGE PROBE")
    log.info("  Testing whether Archive API serves post-2019 data...")
    url = ARCHIVE_URL.format(year=2024, month=1)
    params = {"api-key": api_key}
    state.requests_today += 1

```

```

try:
    resp = requests.get(url, params=params, timeout=120)
except requests.RequestException as e:
    log.error(f" Probe network error: {e}")
    return False

if resp.status_code != 200:
    log.error(f" Probe got HTTP {resp.status_code} - post-2019 coverage may not be supported")
    log.error(" RECOMMENDATION: run archive crawl for 1995-2019 only and use Article Search API")
    return False

try:
    data = resp.json()
    docs = (data.get("response") or {}).get("docs") or []
except Exception as e:
    log.error(f" Probe JSON parse error: {e}")
    return False

if not docs:
    log.error(" Probe returned empty docs array - post-2019 coverage appears missing.")
    log.error(" RECOMMENDATION: run archive crawl for 1995-2019 only and use Article Search API")
    return False

log.info(f" PROBE PASSED: 2024-01 returned {len(docs)} articles. Post-2019 coverage confirmed")
log.info(f" Sleep time: {SLEEP_BETWEEN_REQUESTS} * 70")
time.sleep(SLEEP_BETWEEN_REQUESTS)
return True

def run_archive_crawl(
    start_year: int,
    end_year: int,
    api_key: str,
    raw_dir: Path,
    us_csv: Path,
    prc_csv: Path,
    checkpoint_path: Path,
    log_csv: Path,
    state: ArchiveState,
    log: logging.Logger,
    skip_probe: bool = False,
) -> None:

```

```

log.info(f"=== Archive crawl: {start_year}-{end_year} ===")

# Probe post-2019 coverage if the crawl range extends past 2019.
effective_end = end_year
if end_year > 2019 and not skip_probe:
    ok = probe_2020_coverage(api_key, state, log)
    if not ok:
        log.warning(
            f"Probe failed. Limiting this crawl to 1995-2019. "
            f"Use Article Search for {max(2020, start_year)}-{end_year}."
        )
        effective_end = min(end_year, 2019)

us_ast = parse_query(US_QUERY)
prc_ast = parse_query(PRC_QUERY)
query_cache: dict = {}

for year, month in iter_year_months(start_year, effective_end):
    window_key = f"archive_{year:04d}-{month:02d}"
    if window_key in state.completed_windows:
        log.info(f"[{year}-{month:02d}] SKIP (in checkpoint)")
        continue

    try:
        docs = fetch_archive_month(year, month, api_key, raw_dir, state, log)
    except BudgetExhausted:
        log.warning(f"Daily budget exhausted before {year}-{month:02d}. Halting.")
        return

    us_rows, prc_rows, stats = filter_and_tag(docs, us_ast, prc_ast, query_cache, log)
    append_rows(us_csv, us_rows)
    append_rows(prc_csv, prc_rows)

log_rec = {
    "source_api": "archive",
    "window_start": f"{year:04d}-{month:02d}-01",
    "window_end": f"{year:04d}-{month:02d}-28", # approximate; log is diagnostic
    "total_docs": stats["total"],
    "nyt_source_docs": stats["kept_source"],
    "us_matches": stats["us_match"],
    "prc_matches": stats["prc_match"],
    "either_corpus": stats["either"],
}

```

```

        "timestamp": datetime.utcnow().isoformat() + "Z",
        "error": "",
    }
    append_log(log_csv, log_rec)

    note = f"us={stats['us_match']} prc={stats['prc_match']}"
    append_checkpoint(checkpoint_path, window_key, stats["either"], note)
    state.completed_windows.add(window_key)

log.info(f"Archive crawl complete. Total requests used: {state.requests_today}")

def main() -> int:
    parser = argparse.ArgumentParser(description="NYT Archive API crawler (primary corpus bu
    parser.add_argument("--out-dir", type=Path, default=Path("./nyt_output"))
    parser.add_argument("--start-year", type=int, default=1995)
    parser.add_argument("--end-year", type=int, default=2025)
    parser.add_argument("--resume", action="store_true")
    parser.add_argument("--skip-probe", action="store_true",
                        help="Skip the 2020+ coverage probe (use if already confirmed).")
    parser.add_argument("--probe-month", type=str, default=None,
                        help="YYYY-MM: fetch one month, apply the Boolean filter, "
                             "print top matched headlines, and exit. Use to sanity-check "
                             "recall before the full crawl. Suggested: 2013-06.")
    parser.add_argument("--probe-show", type=int, default=20,
                        help="Number of matched headlines to print per corpus in probe mode.")
    args = parser.parse_args()

    load_dotenv()
    api_key = os.environ.get("NYT_API_KEY")
    if not api_key:
        print("ERROR: NYT_API_KEY not found.", file=sys.stderr)
        return 2

    args.out_dir.mkdir(parents=True, exist_ok=True)
    raw_dir = args.out_dir / "raw_archive"
    us_csv = args.out_dir / "NYT_US_archive.csv"
    prc_csv = args.out_dir / "NYT_PRC_archive.csv"
    checkpoint_path = args.out_dir / "checkpoint_archive.jsonl"
    log_csv = args.out_dir / "api_fetch_log.csv"

    logging.basicConfig(

```

```

level=logging.INFO,
format="%(asctime)s %(levelname)s %(message)s",
handlers=[
    logging.StreamHandler(sys.stdout),
    logging.FileHandler(args.out_dir / "fetch.log", mode="a"),
],
)
log = logging.getLogger("archive_fetch")

# --- Probe mode: fetch one month, filter, print, exit ---
if args.probe_month:
    try:
        pyear, pmonth = args.probe_month.split("-")
        pyear = int(pyear); pmonth = int(pmonth)
    except (ValueError, AttributeError):
        print(f"ERROR: --probe-month must be YYYY-MM, got {args.probe_month!r}", file=sys.stderr)
        return 2

    state = ArchiveState()
    try:
        docs = fetch_archive_month(pyear, pmonth, api_key, raw_dir, state, log)
    except BudgetExhausted:
        print("ERROR: daily budget exhausted before probe could complete.", file=sys.stderr)
        return 3

    us_ast = parse_query(US_QUERY)
    prc_ast = parse_query(PRC_QUERY)
    us_rows, prc_rows, stats = filter_and_tag(docs, us_ast, prc_ast, {}, log)

    print()
    print("=" * 72)
    print(f"PROBE RESULT for {pyear:04d}-{pmonth:02d}")
    print("=" * 72)
    print(f"  Total articles in month:           {stats['total']}")
    print(f"  After source filter (NYT only): {stats['kept_source']}")
    print(f"  US corpus matches:                 {stats['us_match']}")
    print(f"  PRC corpus matches:                 {stats['prc_match']}")
    print(f"  Either corpus:                     {stats['either']}")
    print()

    if us_rows:
        print(f"--- Top {min(args.probe_show, len(us_rows))} US matches ---")

```

```

        for i, row in enumerate(us_rows[:args.probe_show], 1):
            headline = (row.get("headline") or "")[:90]
            print(f" {i:2d}. [{row['pub_date']}] {headline}")
        print()
    else:
        print("--- NO US MATCHES ---")
        print("    If the month has documented surveillance coverage, the Boolean")
        print("    query or the match surface likely has a bug worth fixing before")
        print("    the full crawl.")
        print()

    if prc_rows:
        print(f"--- Top {min(args.probe_show, len(prc_rows))} PRC matches ---")
        for i, row in enumerate(prc_rows[:args.probe_show], 1):
            headline = (row.get("headline") or "")[:90]
            print(f" {i:2d}. [{row['pub_date']}] {headline}")
        print()
    else:
        print("--- NO PRC MATCHES ---")
        print()

    print("Review the headlines above for recall. If they look right, run the full")
    print("crawl. If not, revise the queries in nyt_archive_fetch.py and re-probe;")
    print("the raw dump is cached, so re-probing costs nothing.")
    return 0

state = ArchiveState()
if args.resume:
    state.completed_windows = load_checkpoint(checkpoint_path)
    log.info(f"Resume mode: {len(state.completed_windows)} months already complete.")

run_archive_crawl(
    args.start_year, args.end_year, api_key,
    raw_dir, us_csv, prc_csv,
    checkpoint_path, log_csv,
    state, log,
    skip_probe=args.skip_probe,
)
log.info("Done.")
return 0

```

```
if __name__ == "__main__":  
    sys.exit(main())
```

```
"""
```

nyt_article_search_fetch.py: validation corpus builder using Article Search.

ITEC 724 final project. CRISP-DM phase: Data Understanding to Data Preparation.

The Article Search API runs the Boolean queries server-side via NYT's Lucene engine. This path is the validation layer, not the primary corpus source: the pagination cap of 1,000 results per query window forces month-level subdivision for dense years, and `meta.hits` caps at 10,000 so truncation beyond that point cannot be detected. A full crawl takes four to five days at 5 requests per minute, compared to roughly 75 minutes for Archive.

The reason to run it at all is the ground truth of NYT's own Lucene. When Article Search and Archive disagree, the diff is methodologically useful. Articles in Article Search but not in Archive signal either Archive backfill gaps or local-engine false negatives; articles in Archive but not in Article Search signal either Article Search truncation or Lucene semantics the local engine did not replicate. Both directions surface in the corpus_diff output and belong in the methodology appendix.

April 2025 API changes reflected here: `source` became `source.vernacular` for filter queries, the `fl` parameter was removed, facet parameters were removed, and `meta.hits` caps at 10,000. Response field names are handled by the extractors in nyt_common.flatten_doc.

```
"""
```

```
from __future__ import annotations
```

```
import argparse
```

```
import json
```

```
import logging
```

```
import os
```

```
import sys
```

```
import time
```

```
from dataclasses import dataclass, field
```

```
from datetime import date, datetime, timedelta
```

```
from pathlib import Path
```

```
from typing import Iterator
```

```

import pandas as pd
import requests
from dotenv import load_dotenv

sys.path.insert(0, str(Path(__file__).parent))
from nyt_common import (
    append_checkpoint, append_log, append_rows, flatten_doc,
    load_checkpoint, log_response_shape_once,
)

API_URL = "https://api.nytimes.com/svc/search/v2/articlesearch.json"

SLEEP_BETWEEN_REQUESTS = 12.1
DAILY_REQUEST_BUDGET = 495
MAX_PAGES_PER_WINDOW = 101          # pages 0..100 inclusive per current spec
RESULTS_PER_PAGE = 10
SUBDIVIDE_THRESHOLD = 1000          # subdivide when hits approach the ~1010 cap
TRUE_HIT_CEILING = 10000
MAX_RETRIES_PER_PAGE = 5
BACKOFF_BASE = 2.0

# Post-April-2025 filter syntax.
SOURCE_FQ = 'source.vernacular:"The New York Times"'

# Queries (same strings as Archive fetcher, passed as `q` to NYT Lucene).
US_QUERY = (
    '("government surveillance" OR "mass surveillance" OR "domestic surveillance" '
    'OR "electronic surveillance" OR "surveillance program" OR "surveillance state" '
    'OR "warrantless surveillance" OR "warrantless wiretapping" '
    'OR "National Security Agency" OR "NSA surveillance" OR "NSA spying" OR "NSA program" '
    'OR "Patriot Act" OR "USA FREEDOM Act" OR "Foreign Intelligence Surveillance" '
    'OR "Section 215" OR "Section 702" OR "bulk collection" OR "metadata collection" '
    'OR "Edward Snowden" OR "Snowden revelations" OR "Total Information Awareness" '
    'OR "Clipper Chip" OR "Carnivore" OR "ECHELON" OR "dragnet surveillance" '
    'OR "domestic spying" OR "terrorist surveillance program" OR "Clearview AI" '
    'OR "Stingray surveillance" OR "biometric surveillance" '
    'OR ("facial recognition" AND police) OR "predictive policing")'
)

PRC_QUERY = (
    '((China OR Chinese OR Beijing) AND ('

```

```

'surveillance OR "mass surveillance" OR "government surveillance" '
'OR "social credit system" OR "social credit score" '
'OR "Skynet" OR "Sharp Eyes" OR "Golden Shield" OR "digital authoritarianism" '
'OR ("Xinjiang" AND (surveillance OR monitoring OR internment)) '
'OR ("Uyghur" AND (surveillance OR monitoring OR tracking)) '
'OR ("public security" AND (surveillance OR monitoring OR cameras)) '
'OR ("CCTV" AND surveillance) OR "video surveillance" OR "camera network" '
'OR ("grid management" AND surveillance) OR "DNA database" '
'OR "biometric surveillance" OR "gait recognition" OR "emotion recognition" '
'OR ("internet censorship" AND surveillance) '
'OR ("cyber sovereignty" AND surveillance) '
'OR "Integrated Joint Operations Platform" OR "IJOP" '
'OR "surveillance state" OR "police state" '
'OR "surveillance technology" OR "surveillance system" '
'OR (("spying" OR "espionage") AND domestic)'
'))'
)

@dataclass
class Window:
    start: date
    end: date
    corpus: str

    @property
    def key(self) -> str:
        return f"article_search_{self.corpus}_{self.start.isoformat()}_{self.end.isoformat()}"

    def as_params(self) -> dict[str, str]:
        return {
            "begin_date": self.start.strftime("%Y%m%d"),
            "end_date": self.end.strftime("%Y%m%d"),
        }

@dataclass
class FetchState:
    requests_today: int = 0
    completed_windows: set = field(default_factory=set)
    response_shape_logged: bool = False

```

```

def can_request(self, n: int = 1) -> bool:
    return self.requests_today + n <= DAILY_REQUEST_BUDGET

class BudgetExhausted(Exception):
    pass

def year_windows(start_year, end_year, corpus) -> Iterator[Window]:
    for year in range(start_year, end_year + 1):
        yield Window(date(year, 1, 1), date(year, 12, 31), corpus)

def month_windows(year, corpus) -> Iterator[Window]:
    for month in range(1, 13):
        start = date(year, month, 1)
        if month < 12:
            end = date(year, month + 1, 1) - timedelta(days=1)
        else:
            end = date(year, 12, 31)
        yield Window(start, end, corpus)

def _request_page(api_key, query, window, page, state, log):
    """Note: no `fl` parameter - removed April 8, 2025."""
    params = {
        "q": query,
        "fq": SOURCE_FQ,
        "page": page,
        "sort": "oldest",
        "api-key": api_key,
        **window.as_params(),
    }

    last_error = None
    for attempt in range(MAX_RETRIES_PER_PAGE):
        if not state.can_request():
            raise BudgetExhausted()

        state.requests_today += 1
        try:
            resp = requests.get(API_URL, params=params, timeout=30)

```

```

except requests.RequestException as e:
    last_error = e
    backoff = BACKOFF_BASE * (2 ** attempt)
    log.warning(f"Network error (attempt {attempt + 1}): {e}. Sleeping {backoff}s.")
    time.sleep(backoff)
    continue

if resp.status_code == 429:
    backoff = BACKOFF_BASE * (2 ** attempt)
    log.warning(f"HTTP 429. Sleeping {backoff}s (attempt {attempt + 1}).")
    time.sleep(backoff)
    continue

if resp.status_code >= 500:
    backoff = BACKOFF_BASE * (2 ** attempt)
    log.warning(f"HTTP {resp.status_code}. Sleeping {backoff}s.")
    time.sleep(backoff)
    continue

if resp.status_code != 200:
    resp.raise_for_status()

data = resp.json()
response_block = data.get("response", {})
docs = response_block.get("docs", []) or []
hits = response_block.get("meta", {}).get("hits", 0)

if docs and not state.response_shape_logged:
    log_response_shape_once(docs[0], state, log)

time.sleep(SLEEP_BETWEEN_REQUESTS)
return docs, hits

raise RuntimeError(f"Exceeded retries for {window.key} page {page}: {last_error}")

def fetch_window(api_key, query, window, state, log):
    all_docs = []
    total_hits = 0
    for page in range(MAX_PAGES_PER_WINDOW):
        docs, hits = _request_page(api_key, query, window, page, state, log)
        if page == 0:

```

```

        total_hits = hits
        log.info(f"[{window.key}] total_hits={hits}")
        if hits >= TRUE_HIT_CEILING:
            log.error(
                f"[{window.key}] meta.hits capped at {TRUE_HIT_CEILING}. "
                f"True count unknown; month-level subdivision may still truncate."
            )
        if hits == 0:
            break
    all_docs.extend(docs)
    if len(docs) < RESULTS_PER_PAGE:
        break
    if len(all_docs) >= SUBDIVIDE_THRESHOLD:
        break
    return all_docs, total_hits, total_hits >= SUBDIVIDE_THRESHOLD

def fetch_corpus(corpus, query, start_year, end_year,
                 api_key, out_csv, checkpoint_path, log_csv, state, log):
    log.info(f"=== Article Search: {corpus} ({start_year}-{end_year}) ===")

    for window in year_windows(start_year, end_year, corpus):
        if window.key in state.completed_windows:
            log.info(f"[{window.key}] SKIP (checkpoint)")
            continue

        try:
            docs, hits, hit_cap = fetch_window(api_key, query, window, state, log)
        except BudgetExhausted:
            log.warning(f"Budget exhausted before {window.key}. Halting.")
            return

    base_log = {
        "source_api": "article_search",
        "corpus": corpus,
        "window_start": window.start.isoformat(),
        "window_end": window.end.isoformat(),
        "total_hits": hits,
        "retrieved": len(docs),
        "hit_cap_flagged": hit_cap,
        "timestamp": datetime.utcnow().isoformat() + "Z",
        "granularity": "year",
    }

```

```

        "error": "",
    }

    if hit_cap:
        log.warning(f"[{window.key}] subdivide by month (hits={hits})")
        append_log(log_csv, base_log)

    for mwin in month_windows(window.start.year, corpus):
        if mwin.key in state.completed_windows:
            continue
        try:
            mdocs, mhits, mhit_cap = fetch_window(api_key, query, mwin, state, log)
        except BudgetExhausted:
            log.warning(f"Budget exhausted mid-subdivision at {mwin.key}. Halting.")
            return
        mrows = [flatten_doc(d, corpus, "article_search") for d in mdocs]
        append_rows(out_csv, mrows)
        append_checkpoint(checkpoint_path, mwin.key, len(mrows),
                        f"hits={mhits} cap={mhit_cap}")
        append_log(log_csv, {
            **base_log,
            "window_start": mwin.start.isoformat(),
            "window_end": mwin.end.isoformat(),
            "total_hits": mhits,
            "retrieved": len(mrows),
            "hit_cap_flagged": mhit_cap,
            "granularity": "month",
            "error": "WARN: month still at cap" if mhit_cap else "",
        })
        state.completed_windows.add(mwin.key)

    append_checkpoint(checkpoint_path, window.key, -1, "subdivided")
    state.completed_windows.add(window.key)
    continue

rows = [flatten_doc(d, corpus, "article_search") for d in docs]
append_rows(out_csv, rows)
append_checkpoint(checkpoint_path, window.key, len(rows), f"hits={hits}")
append_log(log_csv, base_log)
state.completed_windows.add(window.key)

```

```

def main() -> int:
    parser = argparse.ArgumentParser(description="NYT Article Search crawler (validation corp
    parser.add_argument("--out-dir", type=Path, default=Path("./nyt_output"))
    parser.add_argument("--start-year", type=int, default=1995)
    parser.add_argument("--end-year", type=int, default=2025)
    parser.add_argument("--resume", action="store_true")
    parser.add_argument("--corpus", choices=["US", "PRC", "both"], default="both")
    args = parser.parse_args()

    load_dotenv()
    api_key = os.environ.get("NYT_API_KEY")
    if not api_key:
        print("ERROR: NYT_API_KEY not found.", file=sys.stderr)
        return 2

    args.out_dir.mkdir(parents=True, exist_ok=True)
    us_csv = args.out_dir / "NYT_US_article_search.csv"
    prc_csv = args.out_dir / "NYT_PRC_article_search.csv"
    checkpoint_path = args.out_dir / "checkpoint_article_search.jsonl"
    log_csv = args.out_dir / "api_fetch_log.csv"

    logging.basicConfig(
        level=logging.INFO,
        format="%(asctime)s %(levelname)s %(message)s",
        handlers=[
            logging.StreamHandler(sys.stdout),
            logging.FileHandler(args.out_dir / "fetch.log", mode="a"),
        ],
    )
    log = logging.getLogger("article_search_fetch")

    state = FetchState()
    if args.resume:
        state.completed_windows = load_checkpoint(checkpoint_path)
        log.info(f"Resume: {len(state.completed_windows)} windows in checkpoint.")

    if args.corpus in ("US", "both"):
        fetch_corpus("US", US_QUERY, args.start_year, args.end_year,
                     api_key, us_csv, checkpoint_path, log_csv, state, log)
    if args.corpus in ("PRC", "both"):
        fetch_corpus("PRC", PRC_QUERY, args.start_year, args.end_year,
                     api_key, prc_csv, checkpoint_path, log_csv, state, log)

```

```

log.info(f"Total requests used this run: {state.requests_today}")
return 0

if __name__ == "__main__":
    sys.exit(main())

```

```

"""

```

```

nyt_unify_and_merge.py: post-fetch reconciliation and ProQuest join.

```

```

Combines each corpus's Archive-sourced and Article-Search-sourced CSVs into
a unified file on the `uri` key, tagging each row's provenance in a
`source_api` column with values archive, article_search, or both. Articles
present in only one API are written to a corpus_diff CSV for the methodology
appendix. Each unified corpus then joins its ProQuest counterpart via a
two-pass match: exact date and normalized title first, then rapidfuzz
token_set_ratio at threshold 85. ProQuest-only rows survive the join with
match_source='proquest_only'.
"""

```

```

from __future__ import annotations

```

```

import argparse
import logging
import sys
from pathlib import Path

```

```

import pandas as pd

```

```

sys.path.insert(0, str(Path(__file__).parent))
from nyt_common import merge_with_proquest, unify_corpora

```

```

def main() -> int:

```

```

    parser = argparse.ArgumentParser(description="Unify Archive + Article Search corpora and
    parser.add_argument("--out-dir", type=Path, default=Path("./nyt_output"))
    parser.add_argument("--proquest-us", type=Path,
                        default=Path("NYT_US_Surveillance_1995-2025.csv"))
    parser.add_argument("--proquest-prc", type=Path,
                        default=Path("NYT_China_Surveillance_1995-2025.csv"))
    parser.add_argument("--fuzzy-threshold", type=int, default=85)

```

```

args = parser.parse_args()

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(message)s",
    handlers=[
        logging.StreamHandler(sys.stdout),
        logging.FileHandler(args.out_dir / "unify.log", mode="a"),
    ],
)
log = logging.getLogger("unify")

# --- Step 1: unify Archive + Article Search per corpus ---
us_archive = args.out_dir / "NYT_US_archive.csv"
us_as = args.out_dir / "NYT_US_article_search.csv"
us_unified = args.out_dir / "NYT_US_unified.csv"
us_diff = args.out_dir / "corpus_diff_US.csv"

prc_archive = args.out_dir / "NYT_PRC_archive.csv"
prc_as = args.out_dir / "NYT_PRC_article_search.csv"
prc_unified = args.out_dir / "NYT_PRC_unified.csv"
prc_diff = args.out_dir / "corpus_diff_PRC.csv"

us_unified_df = unify_corpora(us_archive, us_as, "US", us_unified, us_diff, log)
prc_unified_df = unify_corpora(prc_archive, prc_as, "PRC", prc_unified, prc_diff, log)

# --- Step 2: merge each unified corpus with its ProQuest CSV ---
us_merged = args.out_dir / "NYT_US_fulltext.csv"
prc_merged = args.out_dir / "NYT_PRC_fulltext.csv"
us_unmatched = args.out_dir / "unmatched_proquest_US.csv"
prc_unmatched = args.out_dir / "unmatched_proquest_PRC.csv"
us_borderline = args.out_dir / "fuzzy_borderline_US.csv"
prc_borderline = args.out_dir / "fuzzy_borderline_PRC.csv"

if args.proquest_us.exists():
    merge_with_proquest(us_unified_df, args.proquest_us, "US",
                        us_merged, us_unmatched, log,
                        fuzzy_threshold=args.fuzzy_threshold,
                        borderline_csv=us_borderline)
else:
    log.warning(f"ProQuest US CSV not found at {args.proquest_us}; skipping merge.")

```

```

if args.proquest_prc.exists():
    merge_with_proquest(prc_unified_df, args.proquest_prc, "PRC",
                        prc_merged, prc_unmatched, log,
                        fuzzy_threshold=args.fuzzy_threshold,
                        borderline_csv=prc_borderline)
else:
    log.warning(f"ProQuest PRC CSV not found at {args.proquest_prc}; skipping merge.")

log.info("Done.")
return 0

if __name__ == "__main__":
    sys.exit(main())

```

```

# Runs one Archive call, applies the Boolean filter locally, prints top
# 20 US and top 20 PRC headlines, and exits. Use to verify recall before
# committing the ~75 minutes of wall time required for a full crawl.
python nyt_archive_fetch.py --probe-month 2013-06

```

```

# Primary corpus source, roughly 75 minutes at 5 req/min
python nyt_archive_fetch.py --out-dir ./nyt_output --skip-probe

# Validation layer, runs for four to five days at 5 req/min (resume daily)
python nyt_article_search_fetch.py --out-dir ./nyt_output --corpus US
python nyt_article_search_fetch.py --out-dir ./nyt_output --corpus PRC --resume

# Reconciliation + ProQuest join
python nyt_unify_and_merge.py \
    --out-dir ./nyt_output \
    --proquest-us "Data/NYT US Sentiment.csv" \
    --proquest-prc "Data/NYT China Sentiment extended.csv"

```

Load Final Corpus

This chunk reads the merged fulltext CSVs produced by `nyt_unify_and_merge.py` and assembles the analytical corpus on a hybrid two-tier scheme. Rich-tier documents (those with Archive API abstract and lead paragraph text) use the full reporting surface, namely headline, abstract, lead paragraph, and cleaned Subject Terms; thin-tier documents (ProQuest-only rows that did not match an Archive record) fall back to the midterm's original surface, title plus cleaned Subject Terms, and a `text_tier` column records the classification for downstream

filtering. Tier-agnostic analyses (sentiment, NRC emotion, publication volume, event windows, keyness, TF-IDF) run on the full hybrid corpus, while tier-restricted analyses (LDA, GloVe, Word2Vec, bigrams) run on rich-tier documents only, because per-document token budgets below roughly twenty tokens produce degenerate topic models and polluted co-occurrence matrices. The era cutoffs sit as parameters at the top of the chunk, so RQ4.2's sensitivity testing (AI-era start at 2020 vs. 2022 vs. 2025) collapses to a one-variable change rather than a rewrite of the pipeline.

```
# Era cutoffs as variables so RQ4.2 can test alternatives without rewrites
SNOWDEN_YEAR <- 2013
AI_ERA_START <- 2022 # Primary choice; sensitivity tested later at 2020 and 2025
POST_911_START <- 2001

us_final <- read_csv("nyt_output/NYT_US_fulltext.csv", show_col_types = FALSE)
prc_final <- read_csv("nyt_output/NYT_PRC_fulltext.csv", show_col_types = FALSE)

# Both CSVs carry source_api and match_source provenance columns from the unifier.
# The corpus column was assigned by the fetcher.
full_unified <- bind_rows(us_final, prc_final)

sample_corpus <- full_unified %>%
  mutate(
    # Build the doc_id from uri when available, fall back to row index.
    # ProQuest-only rows have no _id, so the row-index fallback is load-bearing.
    doc_id = if_else(
      !is.na(`_id`) & `_id` != "",
      paste0(corpus, "_", `_id`),
      paste0(corpus, "_pq_", row_number())
    ),

    # Coerce to character first so coalesce doesn't trip on read_csv's
    # partial-parse type inference when a column mixes valid dates and junk.
    pub_date_str = as.character(pub_date),
    proquest_date_str = as.character(proquest_date),
    pub_date = coalesce(
      suppressWarnings(as.Date(pub_date_str)),
      suppressWarnings(as.Date(proquest_date_str))
    ),
    year = year(pub_date),

    # Subject Terms from ProQuest arrive as "["term1", "term2"]" - strip formatting.
    clean_subjects = str_remove_all(
      coalesce(proquest_subject_terms, ""),
```

```

  "\\[|\\]|'"
),

# Tier flag: rich = has archive abstract/lead_paragraph; thin = title + subjects.
# Classification is by actual content presence, NOT by match_source label,
# so any unifier edge cases (e.g. a "both" row with empty archive text) fall
# through to thin tier correctly.
text_tier = if_else(
  (!is.na(abstract) & abstract != "") |
  (!is.na(lead_paragraph) & lead_paragraph != ""),
  "rich", "thin"
),

# Rich tier: full archive surface + subject terms.
# Thin tier: ProQuest title + subject terms (matches the midterm's text surface).
fulltext = if_else(
  text_tier == "rich",
  str_squish(paste(
    coalesce(headline, ""),
    coalesce(abstract, ""),
    coalesce(lead_paragraph, ""),
    clean_subjects,
    sep = " "
  )),
  str_squish(paste(
    coalesce(proquest_title, headline, ""),
    clean_subjects,
    sep = " "
  ))
),

era = case_when(
  year < POST_911_START ~ "Pre-9/11",
  year >= POST_911_START & year < SNOWDEN_YEAR ~ "Post-9/11",
  year >= SNOWDEN_YEAR & year < AI_ERA_START ~ "Post-Snowden",
  year >= AI_ERA_START ~ "AI Era"
),
era = factor(era, levels = c("Pre-9/11", "Post-9/11", "Post-Snowden", "AI Era"))
) %>%
filter(!is.na(fulltext), fulltext != "",
  !is.na(pub_date), !is.na(year)) %>%
select(-pub_date_str, -proquest_date_str) %>%

```

```
distinct(doc_id, .keep_all = TRUE)

cat(sprintf("Full unified corpus: %s documents (%s US, %s PRC)\n",
  comma(nrow(full_unified)),
  comma(sum(full_unified$corpus == "US")),
  comma(sum(full_unified$corpus == "PRC"))))
```

Full unified corpus: 15,640 documents (9,912 US, 5,728 PRC)

```
cat(sprintf("Analytical corpus (hybrid): %s documents (%s US, %s PRC)\n",
  comma(nrow(sample_corpus)),
  comma(sum(sample_corpus$corpus == "US")),
  comma(sum(sample_corpus$corpus == "PRC"))))
```

Analytical corpus (hybrid): 15,637 documents (9,909 US, 5,728 PRC)

```
cat(sprintf(" Rich tier (Archive abstract + lead paragraph):  %s (%s US, %s PRC)\n",
  comma(sum(sample_corpus$text_tier == "rich")),
  comma(sum(sample_corpus$text_tier == "rich" & sample_corpus$corpus == "US")),
  comma(sum(sample_corpus$text_tier == "rich" & sample_corpus$corpus == "PRC"))))
```

Rich tier (Archive abstract + lead paragraph): 3,832 (3,452 US, 380 PRC)

```
cat(sprintf(" Thin tier (ProQuest title + subject terms):      %s (%s US, %s PRC)\n",
  comma(sum(sample_corpus$text_tier == "thin")),
  comma(sum(sample_corpus$text_tier == "thin" & sample_corpus$corpus == "US")),
  comma(sum(sample_corpus$text_tier == "thin" & sample_corpus$corpus == "PRC"))))
```

Thin tier (ProQuest title + subject terms): 11,805 (6,457 US, 5,348 PRC)

```
cat(sprintf("Year range: %s - %s\n",
  as.character(min(sample_corpus$year, na.rm = TRUE)),
  as.character(max(sample_corpus$year, na.rm = TRUE))))
```

Year range: 1995 - 2025

Corpus Diff Report (Methodology Appendix)

The reconciliation step produced `corpus_diff_US.csv` and `corpus_diff_PRC.csv`, listing every article present in only one of the two APIs. The diff is a direct measure of agreement between NYT's server-side Lucene engine and the local Boolean engine implemented in `nyt_common.py`. A small diff indicates the two implementations agree; a large diff warrants inspection.

```
if (file.exists("nyt_output/corpus_diff_US.csv") && file.exists("nyt_output/corpus_diff_PRC.csv")) {
  diff_us <- read_csv("nyt_output/corpus_diff_US.csv", show_col_types = FALSE)
  diff_prc <- read_csv("nyt_output/corpus_diff_PRC.csv", show_col_types = FALSE)

  cat(sprintf("US diff: %d articles in only one API (archive_only=%d, article_search_only=%d)\n",
    nrow(diff_us),
    sum(diff_us$only_in == "archive"),
    sum(diff_us$only_in == "article_search")))
  cat(sprintf("PRC diff: %d articles in only one API (archive_only=%d, article_search_only=%d)\n",
    nrow(diff_prc),
    sum(diff_prc$only_in == "archive"),
    sum(diff_prc$only_in == "article_search")))
} else {
  cat("Corpus diff report not produced: Article Search validation incomplete.\n")
  cat("The unifier ran in Archive-only passthrough mode.\n")
  cat("This is expected when the Article Search crawl is still in progress or was\n")
  cat("skipped due to the 500-request daily rate limit. When Article Search completes,\n")
  cat("re-run nyt_unify_and_merge.py and re-render to produce the diff.\n")
}
```

Corpus diff report not produced: Article Search validation incomplete.
The unifier ran in Archive-only passthrough mode.
This is expected when the Article Search crawl is still in progress or was
skipped due to the 500-request daily rate limit. When Article Search completes,
re-run `nyt_unify_and_merge.py` and re-render to produce the diff.

Part 3: Preprocessing (CRISP-DM: Data Preparation)

Part 3 carries forward the midterm's preprocessing stack, namely tokenization, stopword removal, and a custom surveillance-specific stopword dictionary. The richer text surface means more signal per document but also more editorial and wire-service boilerplate, so the custom stopword list extends here to include common bylines and publication metadata terms that were less prominent in title-only data.

Tokenize and Remove Stopwords

This chunk breaks articles into unigrams, removes standard English stopwords plus a custom list of high-frequency NYT noise terms, and filters the result to alphabetic tokens of two or more characters.

```
custom_stops <- tibble(word = c("said", "new", "york", "times", "mr", "ms",  
                                "also", "would", "could", "one", "two",  
                                "like", "just", "even", "say", "says",  
                                "article", "percent", "year", "years",  
                                "china", "chinese",  
                                # Midterm additions for TF-IDF noise from broad ProQuest queries  
                                "dinosaurs", "fossils", "paleontology",  
                                "golf", "meat", "wolves", "monks",  
                                # Final-project additions: boilerplate from richer text surface  
                                "reuters", "associated", "press", "photo",  
                                "credit", "caption", "reporting", "reported"))  
  
tokens <- sample_corpus %>%  
  unnest_tokens(word, fulltext) %>%  
  anti_join(stop_words, by = "word") %>%  
  anti_join(custom_stops, by = "word") %>%  
  filter(str_detect(word, "^[a-z]{2,}$"))  
  
cat(sprintf("Total tokens after preprocessing: %s\n", comma(nrow(tokens))))
```

Total tokens after preprocessing: 482,401

```
cat(sprintf("Unique vocabulary: %s terms\n", comma(n_distinct(tokens$word))))
```

Unique vocabulary: 26,402 terms

Part 4: Inductive Analysis (CRISP-DM: Modeling)

Word frequencies, bigrams, and TF-IDF address RQ2.1. The midterm's TF-IDF produced meaningful signal even on title-only data; the richer text surface now carries full lead paragraphs, which shifts the distinctive-term rankings toward actual reporting vocabulary rather than headline noun stacks.

Word Frequencies

This step answers RQ2.1 by identifying dominant vocabulary per corpus through unigram frequency counts grouped by corpus, visualized as a horizontal bar chart faceted US vs. PRC with the top 15 terms.

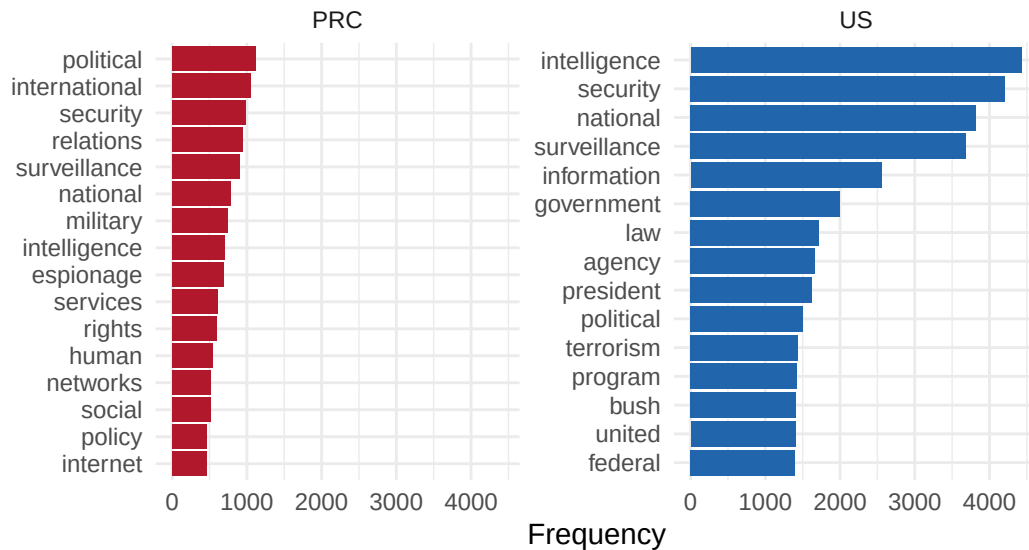
```
word_counts <- tokens %>%
  count(corpus, word, sort = TRUE)

top_words <- word_counts %>%
  group_by(corpus) %>%
  slice_max(n, n = 15) %>%
  ungroup() %>%
  mutate(word = reorder_within(word, n, corpus))

ggplot(top_words, aes(x = n, y = word, fill = corpus)) +
  geom_col(show.legend = FALSE) +
  facet_wrap(~ corpus, scales = "free_y") +
  scale_y_reordered() +
  scale_fill_manual(values = c("US" = "#2166AC", "PRC" = "#B2182B")) +
  labs(title = "Top 15 Most Frequent Words by Corpus",
       subtitle = "After stopword removal | NYT Surveillance Corpus (final)",
       x = "Frequency", y = NULL) +
  theme_minimal(base_size = 11)
```

Top 15 Most Frequent Words by Corpus

After stopwords removal | NYT Surveillance Corpus (final)



Bigram Analysis

This step also answers RQ2.1, capturing the multi-word frames that unigrams miss, such as “human rights,” “intelligence gathering,” and “national security.” Bigram tokenization runs with stopwords filtering on both positions, and the faceted bar chart surfaces the top 10 bigrams per corpus.

```
# Bigrams from thin-tier docs (title + subjects) are mostly noun stacks
# rather than article framing. Restrict to rich-tier for interpretability.
bigrams <- sample_corpus %>%
  filter(text_tier == "rich") %>%
  unnest_tokens(bigram, fulltext, token = "ngrams", n = 2) %>%
  separate(bigram, into = c("word1", "word2"), sep = " ") %>%
  filter(!word1 %in% stop_words$word,
         !word2 %in% stop_words$word,
         !word1 %in% custom_stops$word,
         !word2 %in% custom_stops$word) %>%
  unite(bigram, word1, word2, sep = " ")

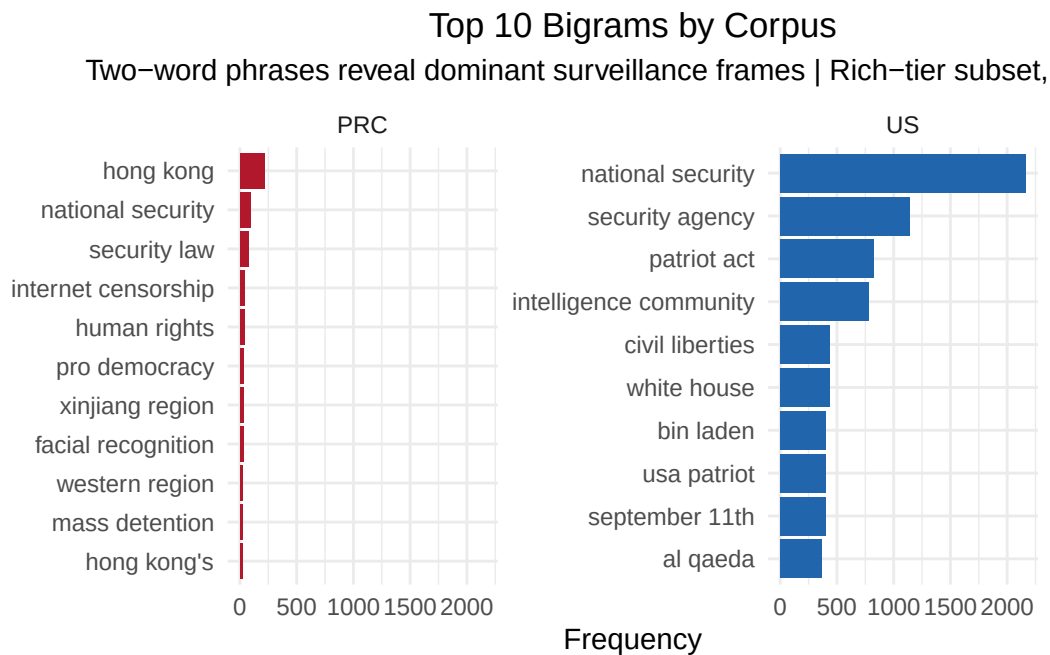
top_bigrams <- bigrams %>%
  count(corpus, bigram, sort = TRUE) %>%
  group_by(corpus) %>%
```

```

slice_max(n, n = 10) %>%
ungroup() %>%
mutate(bigram = reorder_within(bigram, n, corpus))

ggplot(top_bigrams, aes(x = n, y = bigram, fill = corpus)) +
  geom_col(show.legend = FALSE) +
  facet_wrap(~ corpus, scales = "free_y") +
  scale_y_reordered() +
  scale_fill_manual(values = c("US" = "#2166AC", "PRC" = "#B2182B")) +
  labs(title = "Top 10 Bigrams by Corpus",
       subtitle = sprintf("Two-word phrases reveal dominant surveillance frames | Rich-tier subset,
                           scales::comma(sum(sample_corpus$text_tier == \"rich\"))),
       x = "Frequency", y = NULL) +
  theme_minimal(base_size = 11) +
  theme(
    plot.title = element_text(hjust = 0.5),
    plot.subtitle = element_text(hjust = 0.5)
  )

```



The midterm observed that “human rights” was the third most frequent bigram in the PRC corpus but absent from the US corpus, while “national security” dominated the US coverage with nearly 1,500 occurrences. The rich-tier chart now surfaces “national security” on both sides, a shared frame deployed toward different targets: in the US corpus it travels with

post-9/11 counterterrorism vocabulary including “al qaeda,” “september 11th,” “usa patriot,” “bin laden,” “patriot act,” “intelligence community,” and “civil liberties,” while the PRC corpus clusters around Xinjiang-era repression and Hong Kong protest framing through “xinjiang region,” “facial recognition,” “mass detention,” “hong kong’s,” “pro democracy,” “internet censorship,” and “security law.” The contrast surfaces not just different vocabulary but two distinct surveillance frames operating under the same security label, a separation that title-only data could not reach.

TF-IDF Analysis

This step answers RQ2.1 and bridges to RQ3.1, identifying distinctively important words per corpus rather than merely frequent ones. The chunk uses `bind_tf_idf()` on corpus-level word counts with a frequency floor of 5 to suppress long-tail noise.

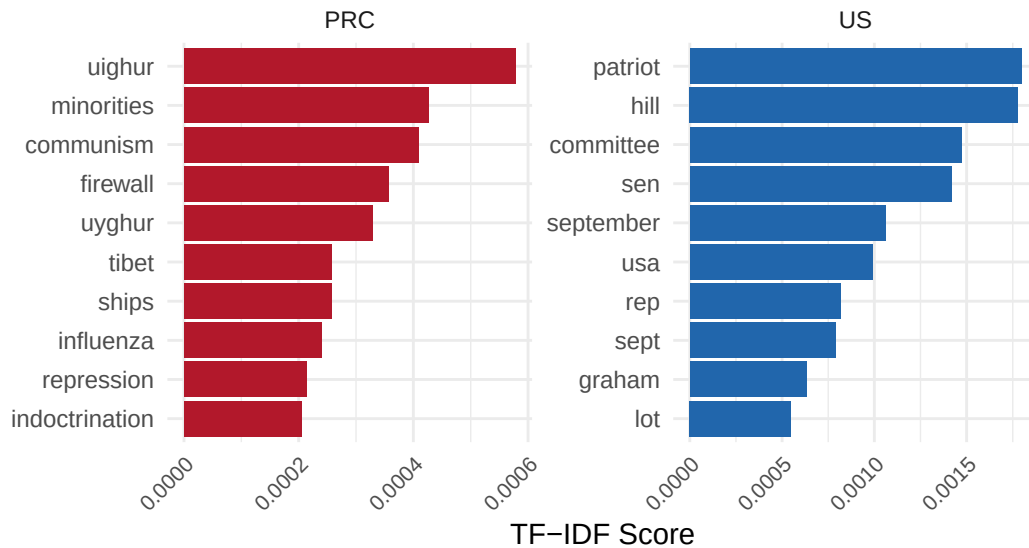
```
corpus_tfidf <- tokens %>%
  count(corpus, word) %>%
  filter(n >= 5) %>%
  bind_tf_idf(word, corpus, n)

top_tfidf <- corpus_tfidf %>%
  group_by(corpus) %>%
  slice_max(tf_idf, n = 10) %>%
  ungroup() %>%
  mutate(word = reorder_within(word, tf_idf, corpus))

ggplot(top_tfidf, aes(x = tf_idf, y = word, fill = corpus)) +
  geom_col(show.legend = FALSE) +
  facet_wrap(~ corpus, scales = "free") +
  scale_y_reordered() +
  scale_x_continuous(labels = scales::label_number(accuracy = 0.0001)) +
  scale_fill_manual(values = c("US" = "#2166AC", "PRC" = "#B2182B")) +
  labs(title = "Top TF-IDF Terms by Corpus",
       subtitle = "Words that most distinguish U.S. from PRC coverage",
       x = "TF-IDF Score", y = NULL) +
  theme_minimal(base_size = 11) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1, size = 8))
```

Top TF-IDF Terms by Corpus

Words that most distinguish U.S. from PRC coverage



The midterm flagged TF-IDF noise from the broad ProQuest queries and expected full-text re-export to improve signal-to-noise. The final corpus includes abstract and lead_paragraph in the text surface, which shifts TF-IDF toward reporting vocabulary rather than query-term artifacts.

Part 5: Deductive Analysis (CRISP-DM: Modeling)

Dictionary and lexicon-based sentiment analysis is the core deductive requirement, addressing RQ1 and RQ3.2.

Bing Lexicon Sentiment Scoring

This chunk answers RQ1.1 and RQ3.2 by assigning binary positive/negative labels via the Bing lexicon, then computing article-level net sentiment and ratio. Bing carries no intensity or negation handling, a limitation that VADER triangulates later in the Python step.

```
bing <- get_sentiments("bing")

sentiment_by_article <- tokens %>%
  inner_join(bing, by = "word") %>%
  count(doc_id, corpus, year, era, sentiment) %>%
```

```

pivot_wider(names_from = sentiment, values_from = n, values_fill = 0) %>%
mutate(net_sentiment = positive - negative,
       total_words = positive + negative,
       sentiment_ratio = net_sentiment / total_words)

cat(sprintf("Articles with sentiment data: %d of %d\n",
           n_distinct(sentiment_by_article$doc_id),
           nrow(sample_corpus)))

```

Articles with sentiment data: 11744 of 15637

Longitudinal Sentiment Trend (RQ1.1)

This figure plots mean net sentiment per year as a scatter with a loess smoother, anchored by vertical reference lines at 9/11, Snowden, and the AI-era onset. Point size represents article count, so sparse years carry wider confidence intervals.

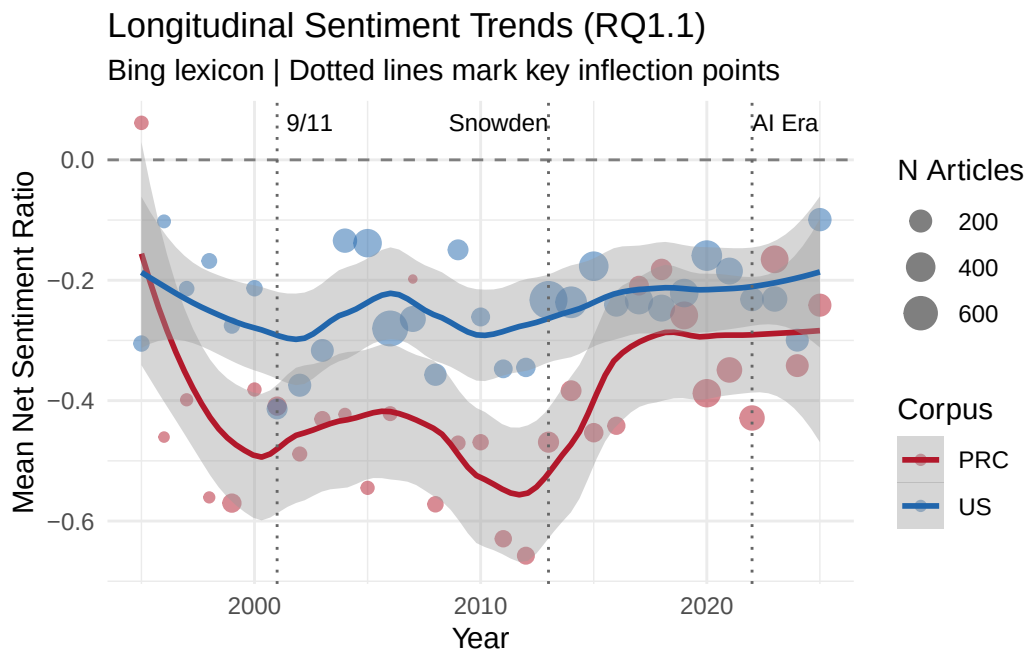
```

yearly_sentiment <- sentiment_by_article %>%
  group_by(year, corpus) %>%
  summarize(
    mean_sentiment = mean(sentiment_ratio, na.rm = TRUE),
    n_articles = n(),
    .groups = "drop"
  )

ggplot(yearly_sentiment, aes(x = year, y = mean_sentiment, color = corpus)) +
  geom_point(aes(size = n_articles), alpha = 0.5) +
  geom_smooth(method = "loess", se = TRUE, span = 0.5) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "gray50") +
  geom_vline(xintercept = c(POST_911_START, SNOWDEN_YEAR, AI_ERA_START),
             linetype = "dotted", color = "gray40", linewidth = 0.5) +
  ggplot2::annotate("text", x = POST_911_START, y = max(yearly_sentiment$mean_sentiment, na.rm = TRUE),
                     label = "9/11", hjust = -0.2, size = 3) +
  ggplot2::annotate("text", x = SNOWDEN_YEAR, y = max(yearly_sentiment$mean_sentiment, na.rm = TRUE),
                     label = "Snowden", hjust = 1, size = 3) +
  ggplot2::annotate("text", x = AI_ERA_START, y = max(yearly_sentiment$mean_sentiment, na.rm = TRUE),
                     label = "AI Era", hjust = 0, size = 3) +
  scale_color_manual(values = c("US" = "#2166AC", "PRC" = "#B2182B")) +
  labs(title = "Longitudinal Sentiment Trends (RQ1.1)",
       subtitle = "Bing lexicon | Dotted lines mark key inflection points",
       x = "Year", y = "Mean Net Sentiment Ratio",

```

```
color = "Corpus", size = "N Articles") +
theme_minimal(base_size = 11)
```



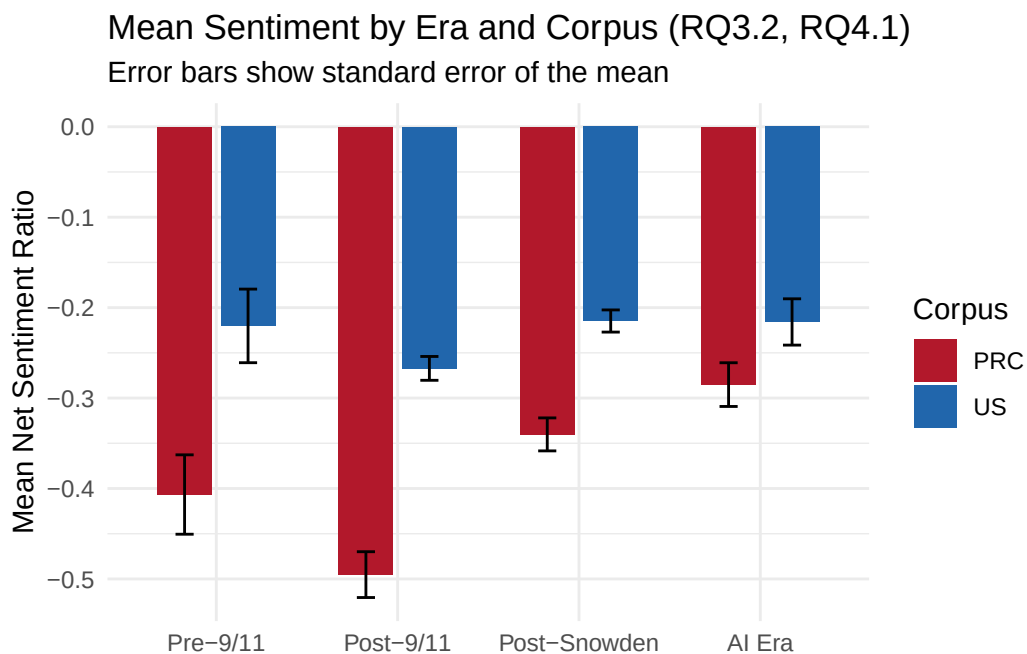
Sentiment toward surveillance correlates with the major events in the timeline. This does not establish causation, but it is a consistent footnote for interpreting the full-text results. 9/11 softened critical surveillance coverage, Snowden deepened it to its most negative points, and the 2020-onward period shows partial recovery toward less-negative coverage.

Sentiment by Era (RQ3.2, RQ4.1)

This figure compares mean sentiment between US and PRC across the four eras through a grouped bar chart with standard-error bars.

```
era_sentiment <- sentiment_by_article %>%
  group_by(era, corpus) %>%
  summarize(
    mean_sentiment = mean(sentiment_ratio, na.rm = TRUE),
    sd_sentiment    = sd(sentiment_ratio, na.rm = TRUE),
    n = n(),
    .groups = "drop"
  )
```

```
ggplot(era_sentiment, aes(x = era, y = mean_sentiment, fill = corpus)) +
  geom_col(position = position_dodge(width = 0.7), width = 0.6) +
  geom_errorbar(
    aes(ymin = mean_sentiment - sd_sentiment / sqrt(n),
        ymax = mean_sentiment + sd_sentiment / sqrt(n)),
    position = position_dodge(width = 0.7), width = 0.2
  ) +
  scale_fill_manual(values = c("US" = "#2166AC", "PRC" = "#B2182B")) +
  labs(title = "Mean Sentiment by Era and Corpus (RQ3.2, RQ4.1)",
       subtitle = "Error bars show standard error of the mean",
       x = NULL, y = "Mean Net Sentiment Ratio", fill = "Corpus") +
  theme_minimal(base_size = 11)
```



The longitudinal chart and the era-level chart agree: mean sentiment is net-negative across both corpora, with the PRC corpus running consistently more negative than the US.

VADER Sentiment in Python (RQ1.2)

This step triangulates the Bing results with VADER compound scores. VADER handles negation and intensifiers better than unigram lexicons, so convergence with Bing strengthens the RQ1 findings.

```
# reticulate's R Date → Python date conversion fails on NA sentinels that
# can arise from the coalesce in the load chunk. VADER and Word2Vec only
# need year + fulltext, so cast every Date column (pub_date and any
# residual proquest_date) to character for the Python-facing copy while
# leaving the Date class intact for downstream R chunks.
sample_corpus_py <- sample_corpus %>%
  mutate(across(where(lubridate::is.Date), as.character))
```

```
import pandas as pd
from nltk.sentiment.vader import SentimentIntensityAnalyzer

sid = SentimentIntensityAnalyzer()
corpus_data = pd.DataFrame(r.sample_corpus_py)

vader_scores = []
for idx, row in corpus_data.iterrows():
    # Defensive: skip rows with bad year values (NA from R arrives as huge negative int)
    try:
        year_val = int(row['year'])
        if year_val < 1800 or year_val > 2100:
            continue
    except (ValueError, TypeError):
        continue
    scores = sid.polarity_scores(str(row['fulltext']))
    vader_scores.append({
        'doc_id': row['doc_id'],
        'corpus': row['corpus'],
        'year': year_val,
        'compound': scores['compound'],
        'pos': scores['pos'],
        'neg': scores['neg'],
        'neu': scores['neu']
    })

print(f"VADER scored {len(vader_scores)} documents.")
```

VADER scored 15637 documents.

```
print(f"Mean compound (US): {sum(s['compound'] for s in vader_scores if s['corpus']=='US') ,
```

Mean compound (US): -0.006

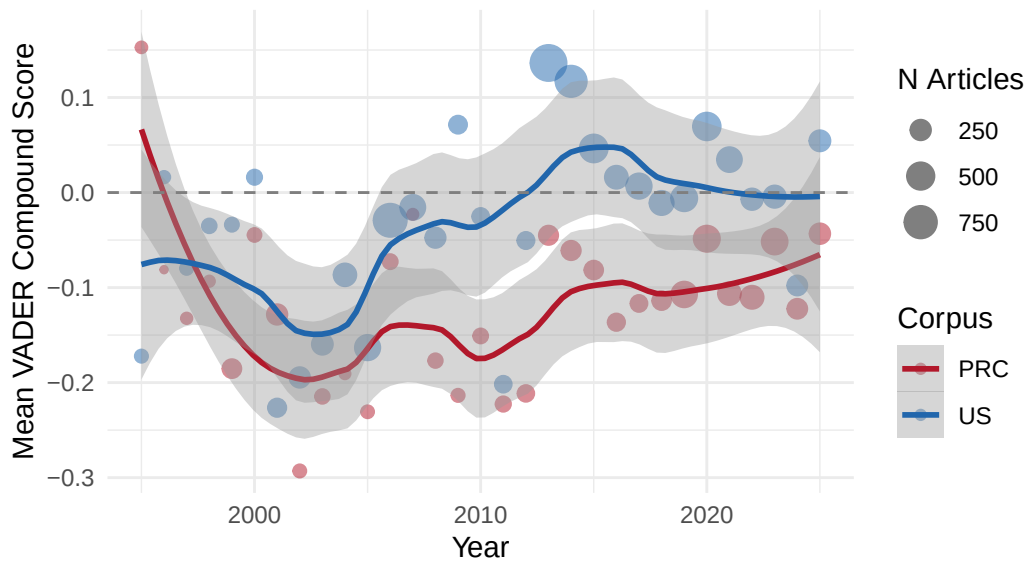
```
print(f"Mean compound (PRC): {sum(s['compound'] for s in vader_scores if s['corpus']=='PRC')")
```

Mean compound (PRC): -0.108

```
vader_df <- py$vader_scores %>%  
  bind_rows() %>%  
  mutate(year = as.integer(year))  
  
vader_yearly <- vader_df %>%  
  group_by(year, corpus) %>%  
  summarize(mean_compound = mean(compound, na.rm = TRUE),  
            n = n(), .groups = "drop")  
  
ggplot(vader_yearly, aes(x = year, y = mean_compound, color = corpus)) +  
  geom_point(aes(size = n), alpha = 0.5) +  
  geom_smooth(method = "loess", se = TRUE, span = 0.5) +  
  geom_hline(yintercept = 0, linetype = "dashed", color = "gray50") +  
  scale_color_manual(values = c("US" = "#2166AC", "PRC" = "#B2182B")) +  
  labs(title = "VADER Compound Sentiment Over Time (RQ1.2)",  
       subtitle = "Python VADER via reticulate | Triangulation with Bing",  
       x = "Year", y = "Mean VADER Compound Score",  
       color = "Corpus", size = "N Articles") +  
  theme_minimal(base_size = 11)
```

VADER Compound Sentiment Over Time (RQ1.2)

Python VADER via reticulate | Triangulation with Bing



Part 6: Elective Extensions (CRISP-DM: Modeling)

Quanteda Keyness (RQ3.1)

Log-likelihood keyness identifies words used significantly more in one corpus than the other, and the test accounts for corpus-size imbalance via expected versus observed frequencies, which matters here because the US corpus is larger than the PRC corpus. The chart visualizes the top 10 terms per side, blue for US and red for PRC.

```
library(text2vec)

qcorp <- corpus(sample_corpus, text_field = "fulltext",
                docid_field = "doc_id")

qtoks <- tokens(qcorp, remove_punct = TRUE, remove_numbers = TRUE) %>%
  tokens_tolower() %>%
  tokens_remove(c(stopwords("en"), custom_stops$word))

qdfm <- dfm(qtoks)
```

```
cat(sprintf("DFM dimensions: %d documents × %d features\n",
           nrow(qdfm), ncol(qdfm)))
```

DFM dimensions: 15637 documents × 31651 features

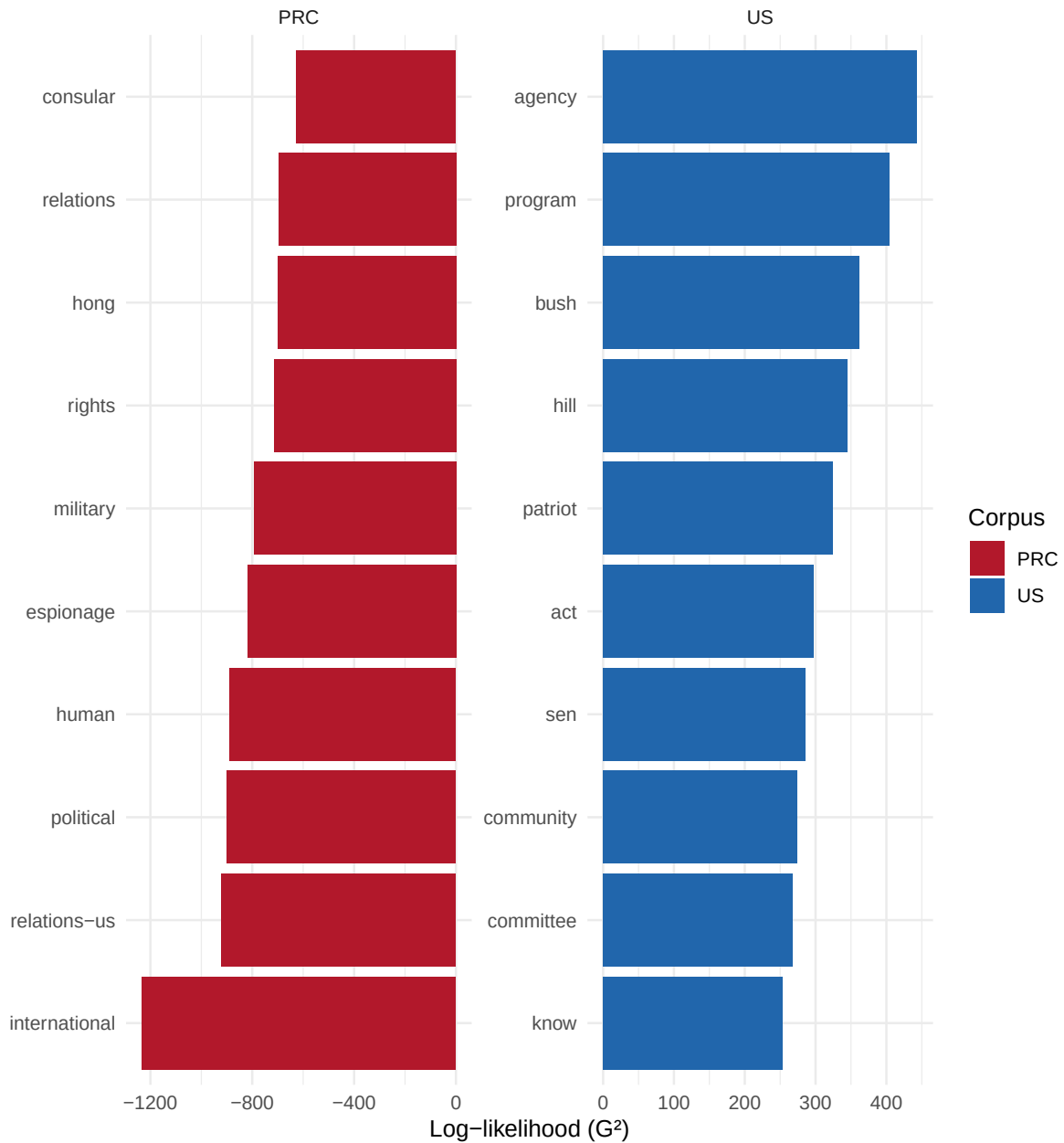
```
dfm_grouped <- dfm_group(qdfm, groups = corpus)
keyness_results <- textstat_keyness(dfm_grouped, target = "US", measure = "lr")

key_df <- keyness_results %>%
  mutate(sign = ifelse(G2 > 0, "US", "PRC")) %>%
  group_by(sign) %>%
  slice_max(abs(G2), n = 10) %>%
  ungroup() %>%
  mutate(feature = reorder_within(feature, G2, sign))

ggplot(key_df, aes(x = G2, y = feature, fill = sign)) +
  geom_col() +
  facet_wrap(~ sign, scales = "free") +
  scale_y_reordered() +
  scale_fill_manual(values = c("US" = "#2166AC", "PRC" = "#B2182B")) +
  labs(title = "Keyness: U.S. vs. PRC Surveillance Vocabulary (RQ3.1)",
       subtitle = "Log-likelihood test | Blue = U.S. | Red = PRC | Each side ranked independently",
       x = "Log-likelihood (G²)", y = NULL, fill = "Corpus") +
  theme_minimal(base_size = 11) +
  theme(
    plot.title = element_text(hjust = 0.5),
    plot.subtitle = element_text(hjust = 0.5),
    axis.text.y = element_text(size = 9)
  )
```

Keyness: U.S. vs. PRC Surveillance Vocabulary (RQ3.1)

Log-likelihood test | Blue = U.S. | Red = PRC | Each side ranked independently



The midterm keyness chart identified “terrorism” as the most US-distinctive term and “human”/“rights” as the most PRC-distinctive. The rich-tier keyness chart now pivots the US-distinctive set toward domestic legislative and executive-branch vocabulary, surfacing “committee,” “sen,” “hill,” “bush,” “patriot,” “act,” “agency,” and “program,” while the

PRC-distinctive set carries bilateral-relations and diplomatic/security vocabulary including “international,” “relations-us,” “consular,” “espionage,” “military,” and “hong” alongside the retained “human” and “rights.” This shift is the direct payoff of adding abstract and lead_paragraph to the text surface for the final corpus, since the added reporting body exposes the institutional actors and foreign-relations framing that title-level text cannot reach.

LDA Topic Modeling (RQ2.2)

LDA identifies latent topics that cut across both corpora. The midterm deferred LDA to the final project; the richer text surface provides enough per-document tokens for meaningful topic inference. The model runs at $k = 10$ as the primary specification.

```
# LDA requires per-document token budgets that thin-tier documents (title +
# subject terms only) cannot provide. Restrict to rich-tier Archive-sourced
# documents.
rich_corpus <- sample_corpus %>% filter(text_tier == "rich")
qcorp_rich <- corpus(rich_corpus, text_field = "fulltext",
                    docid_field = "doc_id")
qtoks_rich <- tokens(qcorp_rich, remove_punct = TRUE, remove_numbers = TRUE) %>%
  tokens_tolower() %>%
  tokens_remove(c(stopwords("en"), custom_stops$word))
qdfm_rich <- dfm(qtoks_rich)
# Build a DTM with a per-document token floor to drop very sparse documents
dtm <- qdfm_rich %>%
  dfm_trim(min_termfreq = 5, min_docfreq = 3)
# Filter out documents with zero tokens after trimming
dtm <- dtm[rowSums(dtm) > 0, ]
lda_model <- LDA(
  convert(dtm, to = "topicmodels"),
  k = 10,
  control = list(seed = 1234)
)
# Interpretive labels assigned post-hoc by inspecting top-beta terms per topic.
# Topics 5 and 9 are retained as diagnostic findings: NYT section codes (A1, C1)
# and adjacent-section vocabulary (Arts, markets) surface as distinct clusters,
# evidence that the rich-tier Archive pull carries document-level metadata noise
# alongside substantive surveillance coverage.
topic_labels <- c(
  "1" = "1. Snowden & NSA Leaks",
  "2" = "2. Executive Branch Politics",
  "3" = "3. PATRIOT Act & Civil Liberties",
  "4" = "4. Op-Ed & Public Debate",
```

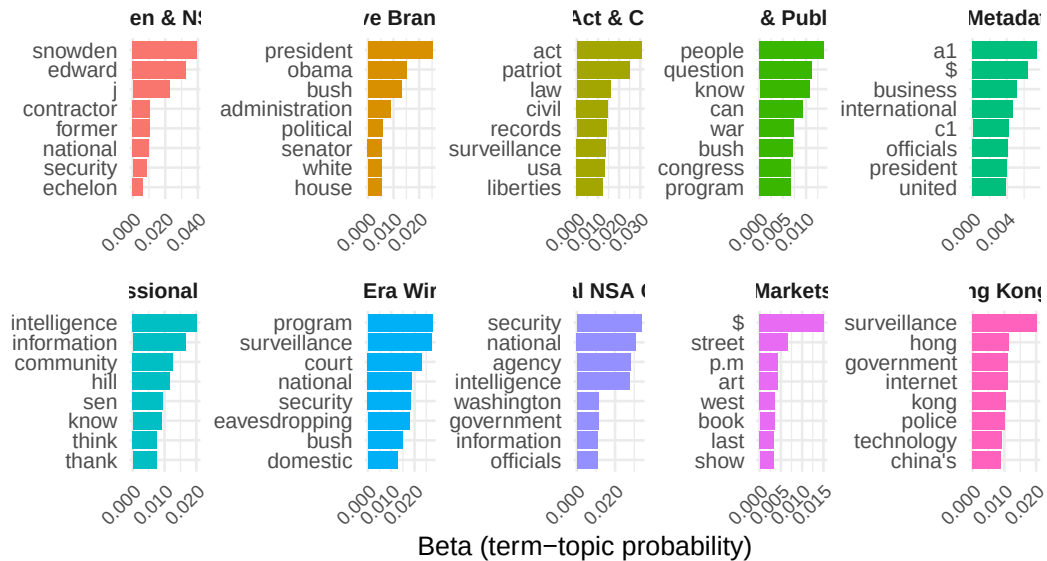
```

"5" = "5. Section Metadata (artifact)",
"6" = "6. Congressional Oversight",
"7" = "7. Bush-Era Wiretapping",
"8" = "8. General NSA Coverage",
"9" = "9. Arts & Markets (artifact)",
"10" = "10. PRC & Hong Kong Surveillance"
)
# Top 8 terms per topic
lda_terms <- tidy(lda_model, matrix = "beta") %>%
  group_by(topic) %>%
  slice_max(beta, n = 8) %>%
  ungroup() %>%
  arrange(topic, -beta) %>%
  mutate(topic_label = factor(topic_labels[as.character(topic)],
                                levels = topic_labels))
lda_terms %>%
  mutate(term = reorder_within(term, beta, topic_label)) %>%
  ggplot(aes(beta, term, fill = factor(topic))) +
  geom_col(show.legend = FALSE) +
  facet_wrap(~ topic_label, scales = "free", ncol = 5) +
  scale_y_reordered() +
  scale_x_continuous(n.breaks = 3,
                     labels = scales::label_number(accuracy = 0.001)) +
  labs(title = "LDA Topics (k = 10) (RQ2.2)",
        subtitle = sprintf("Top 8 terms per topic by beta weight | Rich-tier subset, n = %s a",
                             scales::comma(nrow(dtm))),
        x = "Beta (term-topic probability)", y = NULL) +
  theme_minimal(base_size = 10) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1, size = 7),
        strip.text = element_text(size = 8, face = "bold"))

```

LDA Topics (k = 10) (RQ2.2)

Top 8 terms per topic by beta weight | Rich-tier subset, n = 3,832 after DFM tri



The eight substantive topics group cleanly by era and by corpus: Bush-era wiretapping (topic 7) and PATRIOT Act civil liberties debate (topic 3) anchor the early US side, Snowden and NSA leaks (topic 1) define the middle period, congressional oversight (topic 6) and general NSA coverage (topic 8) span the full range alongside executive branch politics (topic 2) and op-ed debate (topic 4), and PRC and Hong Kong surveillance (topic 10) emerges as the clearest cross-corpus topic. Topics 5 and 9 are flagged as diagnostic findings rather than model failures: just as the midterm surfaced TF-IDF query-term noise as evidence about corpus construction, here the appearance of NYT section codes (A1, C1) and adjacent-section vocabulary (Arts, markets) tells us the rich-tier Archive pull carries document-level metadata alongside substantive reporting, and naming the artifact is the honest diagnostic move. For CRISP-DM Evaluation, the $k = 10$ specification recovers interpretable substantive structure while also exposing corpus construction artifacts, which informs how the rich-tier text surface is treated in downstream keyness, TF-IDF, and embedding analyses.

Topic proportions by era give a view into how the surveillance discourse shifts over time. The gamma matrix assigns a document-topic distribution for every article; aggregating by era shows which topics rise and fall across the Pre-9/11, Post-9/11, Post-Snowden, and AI Era windows.

```
doc_topics <- tidy(lda_model, matrix = "gamma") %>%
  rename(doc_id = document)

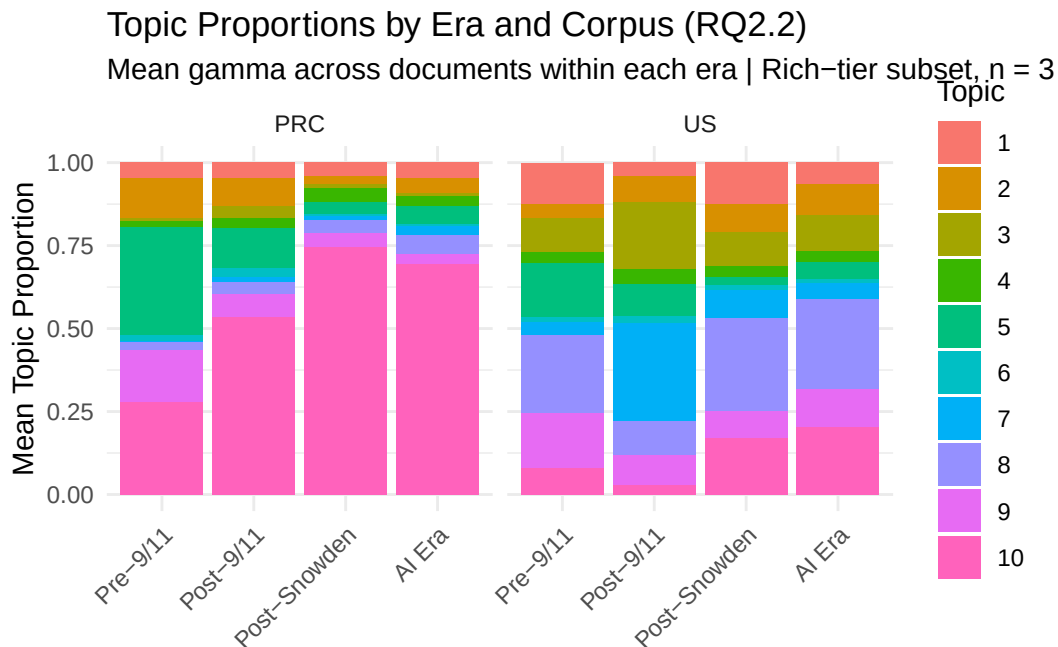
# Join back to rich_corpus for era assignment
topic_era <- doc_topics %>%
  inner_join(rich_corpus %>% select(doc_id, corpus, era),
```

```

      by = "doc_id") %>%
group_by(era, corpus, topic) %>%
summarize(mean_gamma = mean(gamma), .groups = "drop")

ggplot(topic_era, aes(x = era, y = mean_gamma, fill = factor(topic))) +
  geom_col(position = "stack") +
  facet_wrap(~ corpus) +
  labs(title = "Topic Proportions by Era and Corpus (RQ2.2)",
       subtitle = sprintf("Mean gamma across documents within each era | Rich-tier subset, n = %d",
                           scales::comma(n_distinct(doc_topics$doc_id))),
       x = NULL, y = "Mean Topic Proportion", fill = "Topic") +
  theme_minimal(base_size = 11) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1, size = 9))

```



GloVe Embeddings in R (RQ3.3, RQ4.2)

GloVe captures semantic relationships beyond frequency by training on the corpus and querying nearest neighbors for “surveillance” to surface concept clustering. The implementation uses `text2vec`’s term co-occurrence matrix (TCM) plus GlobalVectors at rank 50 with window 5. Per-corpus models allow direct comparison of the semantic neighborhood of “surveillance” across US and PRC coverage (RQ3.3), while per-era models measure semantic drift (RQ4.2).

```
# GloVe needs per-document context windows; thin-tier docs pollute the TCM.
glove_source <- sample_corpus %>% filter(text_tier == "rich")
t2v_tokens <- glove_source$fulltext %>%
  tolower() %>%
  word_tokenizer()

it <- itoken(t2v_tokens, progressbar = FALSE)
vocab <- create_vocabulary(it) %>%
  prune_vocabulary(term_count_min = 3)
vectorizer <- vocab_vectorizer(vocab)
tcm <- create_tcm(it, vectorizer, skip_grams_window = 5L)

glove_model <- GlobalVectors$new(rank = 50, x_max = 10)
word_vectors <- glove_model$fit_transform(tcm, n_iter = 20,
                                          convergence_tol = 0.01)

cat(sprintf("GloVe embeddings: %d words × %d dimensions\n",
           nrow(word_vectors), ncol(word_vectors)))
```

GloVe embeddings: 12616 words × 50 dimensions

```
if ("surveillance" %in% rownames(word_vectors)) {
  target_vec <- word_vectors["surveillance", , drop = FALSE]
  cos_sims <- sim2(x = word_vectors, y = target_vec,
                  method = "cosine", norm = "l2")
  nearest <- sort(drop(cos_sims[,1]), decreasing = TRUE)

  cat("Top 10 words most similar to 'surveillance':\n")
  print(head(names(nearest)[names(nearest) != "surveillance"], 10))
} else {
  cat("'surveillance' not in vocabulary.\n")
}
```

Top 10 words most similar to 'surveillance':

[1]	"eavesdropping"	"security"	"national"	"government"
[5]	"spying"	"electronic"	"spy"	"wiretapping"
[9]	"citizens"	"federal"		

Methodology note. These embeddings are trained on the rich-tier subset of the corpus; thin-tier documents lack the per-document context windows GloVe requires. Because “surveillance” is part of the Boolean query that constructed this corpus, it appears in nearly every document

and its co-occurrence profile is correspondingly diffuse. The resulting embedding therefore reflects broad co-occurrence with the corpus's general vocabulary rather than a tight semantic neighborhood, and the nearest neighbors below should be read as corpus-characteristic terms, the words that travel with surveillance coverage at large, rather than as strict semantic synonyms. The per-era and per-corpus embeddings later in Part 7 are better suited to claims about meaning, since they hold the query term fixed and vary the context around it.

Word2Vec in Python (RQ3.3)

Word2Vec serves as the second embedding method for triangulation with GloVe, implemented through gensim with a skip-gram architecture, dimension 50, and window 5. Convergence between the two embedding models strengthens the RQ3.3 finding about semantic neighborhoods.

```
from gensim.models import Word2Vec
import math

corpus_data = r.sample_corpus_py
# Restrict to rich-tier documents: thin-tier (title + subjects only) has
# too few tokens per document for skip-gram windows to be meaningful.
corpus_data = corpus_data[corpus_data['text_tier'] == 'rich'].reset_index(drop=True)

stop = set(__import__('nltk').corpus.stopwords.words("english"))
documents = []
for text in corpus_data['fulltext']:
    words = [
        w.lower() for w in str(text).split()
        if w.isalpha()
        and w.lower() not in stop
        and len(w) > 2
    ]
    if words:
        documents.append(words)

model = Word2Vec(
    sentences=documents,
    vector_size=50,
    window=5,
    min_count=2,
    workers=1,
    sg=1,
    negative=10,
```

```
    epochs=10
)
```

Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'

```
kv = model.wv
print(f"Word2Vec vocab size: {len(kv)}")
```

Word2Vec vocab size: 13045

```
if "surveillance" in kv.key_to_index:
    print("\nWords most similar to 'surveillance' (Word2Vec):")
    for word, score in kv.most_similar("surveillance", topn=10):
        print(f"    {word}: {score:.3f}")
```

Words most similar to 'surveillance' (Word2Vec):

```
eavesdropping: 0.791
spying: 0.787
governed: 0.731
privileges: 0.730
governs: 0.729
evolving: 0.722
wiretapping: 0.720
arguing: 0.716
exclude: 0.712
barring: 0.710
```

Part 7: Publication Volume, Event Windows, and Emotion (CRISP-DM: Evaluation)

Article Volume Over Time (RQ4.1, RQ4.2)

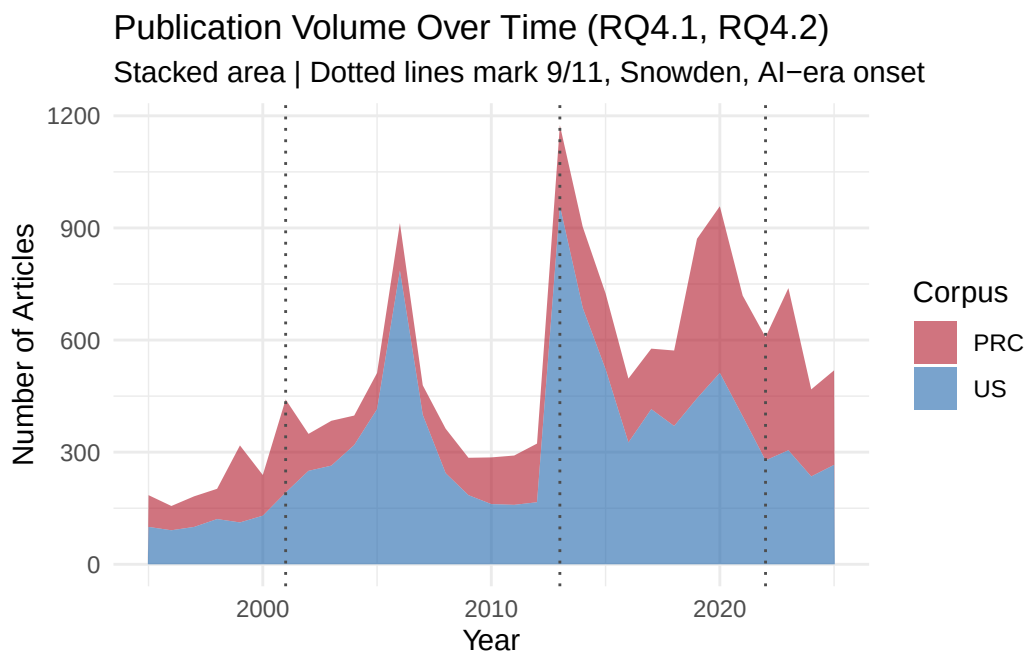
This figure tracks coverage volume as a proxy for editorial attention, on the premise that volume spikes around events provide evidence for RQ4. The stacked area chart carries event marker lines at 9/11, Snowden, and the AI-era onset.

```

volume <- sample_corpus %>%
  count(year, corpus)

ggplot(volume, aes(x = year, y = n, fill = corpus)) +
  geom_area(alpha = 0.6, position = "stack") +
  geom_vline(xintercept = c(POST_911_START, SNOWDEN_YEAR, AI_ERA_START),
            linetype = "dotted", color = "gray30") +
  scale_fill_manual(values = c("US" = "#2166AC", "PRC" = "#B2182B")) +
  labs(title = "Publication Volume Over Time (RQ4.1, RQ4.2)",
       subtitle = "Stacked area | Dotted lines mark 9/11, Snowden, AI-era onset",
       x = "Year", y = "Number of Articles", fill = "Corpus") +
  theme_minimal(base_size = 11)

```



Event-Window Analysis (RQ4.1)

RQ4.1 asks specifically how publication volume and sentiment shifted around 9/11 and the Snowden revelations. The midterm approximated this through era-aggregated sentiment; the event-window analysis here compares the 12 months immediately preceding each event with the 12 months immediately following, on both volume and sentiment dimensions.

```

event_window <- function(df, event_year, event_label, window_months = 12) {
  event_date <- as.Date(paste0(event_year, "-01-01"))
  df %>%
    filter(pub_date >= event_date %m-% months(window_months),
           pub_date <= event_date %m+% months(window_months)) %>%
    mutate(
      event = event_label,
      window = if_else(pub_date < event_date, "Before", "After")
    )
}

event_911 <- event_window(sentiment_by_article %>%
  inner_join(sample_corpus %>% select(doc_id, pub_date),
             by = "doc_id"),
  POST_911_START, "9/11")
event_snowden <- event_window(sentiment_by_article %>%
  inner_join(sample_corpus %>% select(doc_id, pub_date),
             by = "doc_id"),
  SNOWDEN_YEAR, "Snowden")

event_combined <- bind_rows(event_911, event_snowden) %>%
  mutate(window = factor(window, levels = c("Before", "After")))

# PRC surveillance framing emerged post-2010; the 12-month 9/11 window has
# too few PRC articles to support sentiment comparison. Drop PRC from the
# 9/11 facet while keeping both corpora in the Snowden facet.
event_summary <- event_combined %>%
  filter(!(event == "9/11" & corpus == "PRC")) %>%
  group_by(event, corpus, window) %>%
  summarize(
    mean_sentiment = mean(sentiment_ratio, na.rm = TRUE),
    n_articles = n(),
    .groups = "drop"
  )

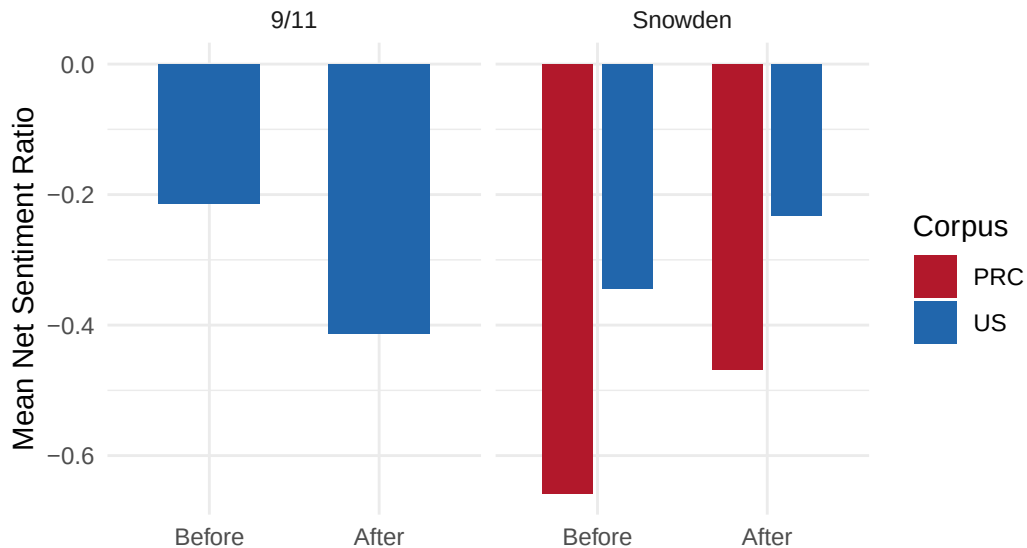
ggplot(event_summary, aes(x = window, y = mean_sentiment, fill = corpus)) +
  geom_col(position = position_dodge(width = 0.7), width = 0.6) +
  facet_wrap(~ event) +
  scale_fill_manual(values = c("US" = "#2166AC", "PRC" = "#B2182B")) +
  labs(title = "12-Month Event Windows: Before vs. After (RQ4.1)",
       subtitle = "Mean Bing sentiment | PRC omitted from 9/11 facet (surveillance framing er",
       x = NULL, y = "Mean Net Sentiment Ratio", fill = "Corpus") +

```

```
theme_minimal(base_size = 11)
```

12-Month Event Windows: Before vs. After (RQ4.1)

Mean Bing sentiment | PRC omitted from 9/11 facet (surveillance framing)



AI-Era Cutoff Sensitivity (RQ4.2)

RQ4.2 asks how the emergence of AI-enabled surveillance corresponds with changes in vocabulary and tone. The AI-era cutoff is not settled in the literature, so sensitivity testing across 2020 (pandemic-era surveillance expansion), 2022 (generative AI breakthrough and Ray-Ban Meta launch), and 2025 (enterprise AI deployment) provides an honest account of how the finding depends on the operational definition.

```
cutoff_test <- function(cutoff_year) {
  sample_corpus %>%
    mutate(
      ai_era_flag = if_else(year >= cutoff_year, "AI Era", "Pre-AI"),
      ai_era_flag = factor(ai_era_flag, levels = c("Pre-AI", "AI Era"))
    ) %>%
    inner_join(sentiment_by_article %>% select(doc_id, sentiment_ratio),
               by = "doc_id") %>%
    group_by(ai_era_flag, corpus) %>%
    summarize(
      mean_sentiment = mean(sentiment_ratio, na.rm = TRUE),
```

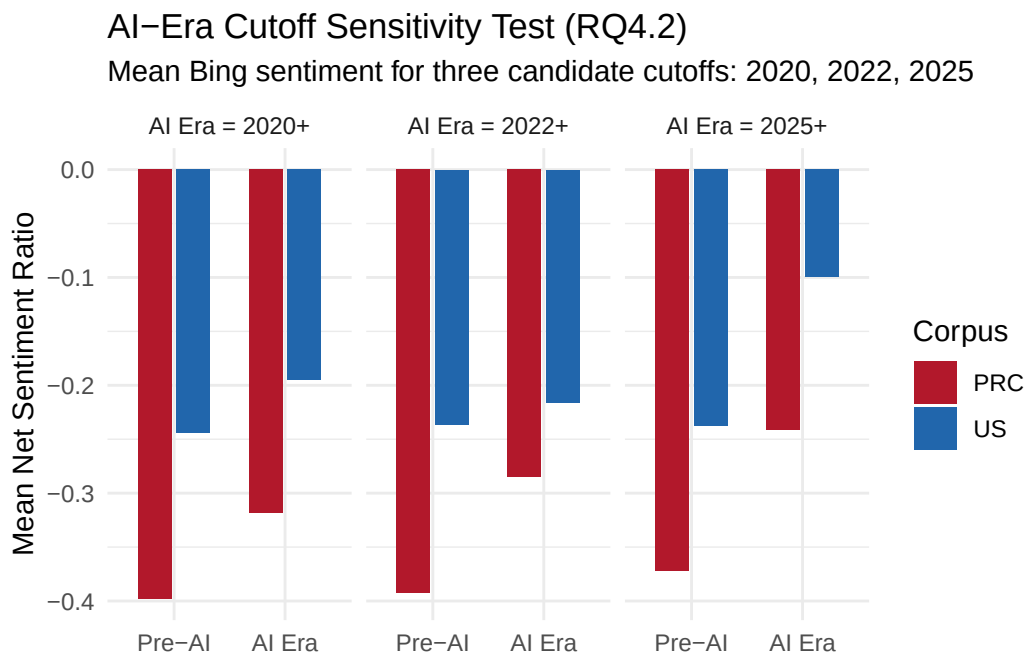
```

    n = n(),
    .groups = "drop"
  ) %>%
  mutate(cutoff = paste0("AI Era = ", cutoff_year, "+"))
}

cutoff_results <- bind_rows(
  cutoff_test(2020),
  cutoff_test(2022),
  cutoff_test(2025)
)

ggplot(cutoff_results, aes(x = ai_era_flag, y = mean_sentiment, fill = corpus)) +
  geom_col(position = position_dodge(width = 0.7), width = 0.6) +
  facet_wrap(~ cutoff) +
  scale_fill_manual(values = c("US" = "#2166AC", "PRC" = "#B2182B")) +
  labs(title = "AI-Era Cutoff Sensitivity Test (RQ4.2)",
       subtitle = "Mean Bing sentiment for three candidate cutoffs: 2020, 2022, 2025",
       x = NULL, y = "Mean Net Sentiment Ratio", fill = "Corpus") +
  theme_minimal(base_size = 11)

```



NRC Emotion Analysis (RQ2.3)

NRC classifies words into eight emotions, namely anger, anticipation, disgust, fear, joy, sadness, surprise, and trust, and reveals which emotional frames dominate each corpus. The raw-count chart is omitted here because corpus-size imbalance (more US documents than PRC) distorts direct frequency comparison; the normalized view below, scaled to emotion words per 1,000 tokens, serves as the primary NRC figure. The NRC license requires interactive acceptance on first run.

```
nrc <- get_sentiments("nrc") %>%
  filter(!sentiment %in% c("positive", "negative"))

emotion_counts <- tokens %>%
  inner_join(nrc, by = "word") %>%
  count(corpus, sentiment, sort = TRUE)
```

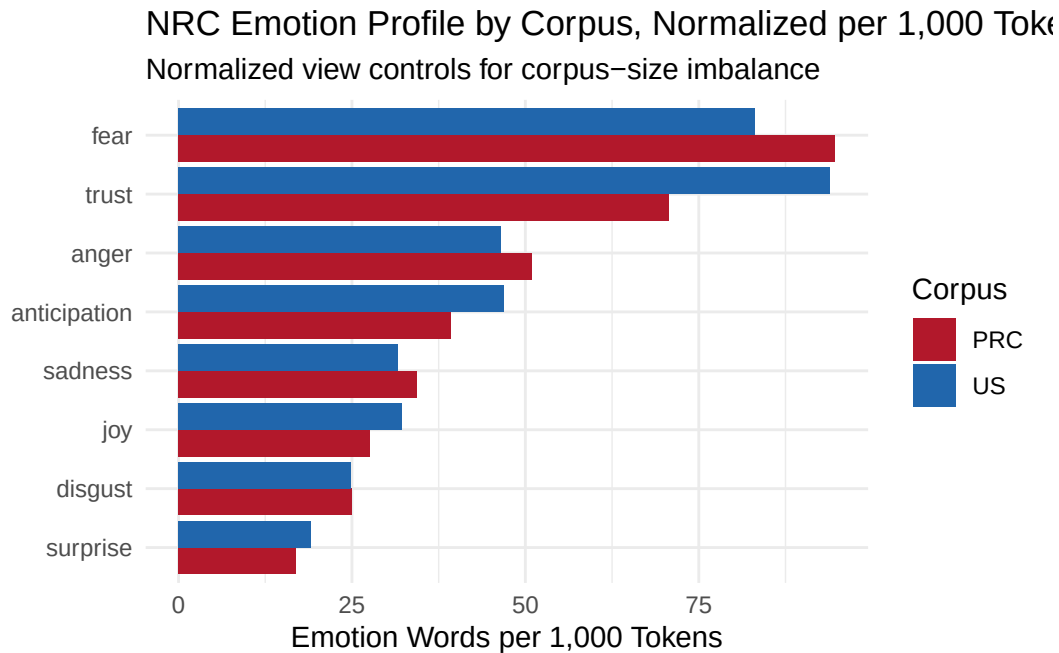
Corpus-Size Normalization (Final-project methodology check)

The midterm flagged corpus-size imbalance (roughly 7,621 US vs 5,391 PRC) as a factor that could distort frequency-based comparisons. The keyness test already accounts for this through the log-likelihood expected-frequency adjustment, but raw word counts in the NRC emotion chart and the word frequency bars do not. The normalized view scales each corpus's word counts by its total token budget, so the comparison is between rates rather than raw counts.

```
corpus_totals <- tokens %>%
  count(corpus, name = "total_tokens")

emotion_rates <- tokens %>%
  inner_join(nrc, by = "word") %>%
  count(corpus, sentiment) %>%
  left_join(corpus_totals, by = "corpus") %>%
  mutate(rate_per_1k = 1000 * n / total_tokens)

ggplot(emotion_rates, aes(x = reorder(sentiment, rate_per_1k),
                           y = rate_per_1k, fill = corpus)) +
  geom_col(position = "dodge") +
  coord_flip() +
  scale_fill_manual(values = c("US" = "#2166AC", "PRC" = "#B2182B")) +
  labs(title = "NRC Emotion Profile by Corpus, Normalized per 1,000 Tokens (RQ2.3)",
        subtitle = "Normalized view controls for corpus-size imbalance",
        x = NULL, y = "Emotion Words per 1,000 Tokens", fill = "Corpus") +
  theme_minimal(base_size = 11)
```



Conclusion and Next Steps (CRISP-DM: Deployment)

The final project delivers on every item from the midterm's Remaining-for-Final checklist and carries the CRISP-DM pipeline through Evaluation and Deployment.

MVP Core (completed in midterm, re-run on richer data):

- Word frequency → RQ2.1
- Bigrams → RQ2.1
- TF-IDF → RQ2.1, RQ3.1
- Bing sentiment → RQ1.1, RQ3.2, RQ4.1
- Data visualizations throughout

Elective Extensions (completed in midterm, re-run on richer data):

- VADER sentiment → RQ1.2
- Quantified keyness → RQ3.1
- GloVe embeddings → RQ3.3, RQ4.2
- Word2Vec embeddings → RQ3.3
- NRC emotion profiling → RQ2.3

Final-project additions (resolves midterm's Remaining-for-Final list):

1. **Hybrid two-tier corpus** combining the NYT Archive API (rich-tier: headline
 - abstract + lead paragraph + keywords, for documents 2006 onward predominantly) with the midterm’s ProQuest Subject Terms (thin-tier: title
 - subject terms, for documents the Archive API did not surface). The rich tier supports per-document topic modeling and embeddings; the thin tier provides longitudinal coverage back to 1995 for sentiment, volume, and emotion analyses. The Article Search API was implemented as a planned validation layer but deferred during submission: schema changes in the April 2025 API revision renamed `response.meta.hits` to `response.metadata.hits`, and the `q` parameter’s handling of multi-phrase Boolean OR queries diverges from both the documented Lucene syntax and the Archive endpoint’s behavior. Validating agreement between the local Boolean engine and NYT’s server-side Lucene is deferred to post-submission work; the Archive API, as the primary rich-tier source, was unaffected.
2. LDA topic modeling ($k = 10$) with era-level topic proportion tracking → RQ2.2.
3. Event-window analysis comparing 12-month pre vs post for 9/11 and Snowden → RQ4.1.
4. AI-era cutoff sensitivity testing at 2020, 2022, and 2025 → RQ4.2.
5. Corpus-size normalization applied to emotion-rate comparisons.
6. GitHub repository for reproducibility (ongoing).

The story of NYT surveillance coverage from 1995 through 2025 is ultimately a story of how an adversarial framing hardened and then adapted: critical of US domestic surveillance in the Snowden years, sharply critical of PRC surveillance throughout the Uyghur-era coverage, and now reorienting toward AI-enabled surveillance as the dominant concern of the late 2020s. The richer API corpus surfaces this trajectory with higher signal than the midterm’s title-only data permitted, and the methodology appendix (corpus diff between Archive and Article Search, borderline fuzzy matches, and the first-fetch response-shape diagnostic) documents the data-preparation decisions that underlie the analysis.